

DENIAL OF ENCRYPTION FROM SECRET SHARING IN DATA

*A Mini Project Report Submitted to
Jawaharlal Nehru Technological University,
in partial fulfilment of the requirements for the award of the degree of*

**BACHELOR OF TECHNOLOGY
IN
COMPUTER SCIENCE AND ENGINEERING
(ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING)**

Submitted By

SRAVANTHI KACHAKAYALWAR

23831A66E3

Under the Guidance of
Mr. Lalendra Kumar Khanday
Assistant Professor
Department of CSE-(AI&ML)



**DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
(ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING)**

GURU NANAK INSTITUTE OF TECHNOLOGY

Ibrahimpattanam, Hyderabad, R. R. District – 501506

May, 2026



GURU NANAK INSTITUTE OF TECHNOLOGY
(Affiliated to JNTUH-Hyderabad)
Ranga Reddy District-501506

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
(ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING)

CERTIFICATE

This is to certify that the Project entitled “**DENIAL OF ENCRYPTION FROM SECRET SHARING IN DATA** ” is being presented with a report by **SRAVANTHI KACHAKAYALWAR (23831A66E3)** in partial fulfilment for the award of the Degree of Bachelor of the Technology in Computer Science and Engineering (Artificial Intelligence and Machine Learning) to the Jawaharlal Nehru Technological University is a record of Bonafide work carried out by them under my guidance and supervision.

The results embodied in this Project Report have not been submitted to any other University or Institute for the award of any Degree or Diploma.

PROJECT GUIDE

Mr. LALENDRA KUMAR KHANDAY

Assistant Professor

PROJECT COORDINATOR

Mr. LALENDRA KUMAR KHANDAY

Assistant Professor

Dr. MAHESH LOKHANDE

HEAD OF THE DEPARTMENT

EXTERNAL EXAMINER

**RAM Innovative Infotech**

M : +91 9581 012 012
E : raminnovativeinfotech@gmail.com
Flat No.#309, Amrutha Ville,
Opp: Yashoda Hospital, Somajiguda,
Hyderabad-82, Telangana, India
WWW.raminnovativeinfotech.com

PROJECT COMPLETION CERTIFICATE

This is to certify that the following student of Third Year B.Tech, Department of
CSE(AI ML) - Guru Nanak Institute of Technology (GNIT) have
completed his/her training and Project at GNIT successfully.

Student Name**Roll No**Sravanthi kachakayalwar23831A66E3

The Training was conducted on Java Full Stack Technology for the
completion of the project titled “ Denial of Encryption from Secret Sharing in Data ”

in 2025-26.

The project has been completed in all aspects.



Mini & Major IEEE Live Projects For
M.E /M.Tech, B.E / B.Tech, MCA, M.S.

DECLARATION BY STUDENTS

I, **Sravanthi Kachakayalwar (23831A66E3)** hereby declare that the mini project entitled “**DENIAL OF ENCRYPTION FROM SECRET SHARING IN DATA**” has been carried out by me as part of the requirements for the award of the **Degree of Bachelor of Technology in the Department of Computer Science and Engineering (Artificial Intelligence and Machine Learning) at Guru Nanak Institute of Technology.**

I confirm the following:

1. The project was undertaken by me under the supervision of our guide, **Mr. Lalendra Kumar Khanday**, from the selection of the topic to the completion of the final report.
2. I have ensured that the results presented in the report are accurate and based on our original work.
3. To the best of our knowledge, the content of this report is free from plagiarism and adheres to ethical standards.
4. Each member of the team has contributed significantly and appropriately to the project work.
5. The project report has been prepared with diligence, ensuring clarity, accuracy, and adherence to academic standards.

I further declare that this report has not been submitted, in part or full, to any other institution or university for the award of any degree or diploma.

Sravanthi Kachakayalwar

23831A66E3

Date:

Place: Ibrahimpatnam, Hyderabad.

DECLARATION BY GUIDE

I, **Mr. Lalendra Kumar Khanday**, hereby declare that I have guided the mini project titled "**DENIAL OF ENCRYPTION FROM SECRET SHARING IN DATA**" Sravanthi Kachakayalwar (23831A66E3). This project was carried out towards the fulfilment of the requirements for the award of the Degree of Bachelor of Technology in Computer Science and Engineering at Guru Nanak Institute of Technology.

As the guide, I confirm the following:

1. I have overseen the entire project process, from the selection of the project title to the submission of the final report.
2. I have reviewed and certified the accuracy and relevance of the results presented in the report.
3. The work carried out is original, free from plagiarism, and adheres to ethical guidelines.
4. The contributions of each student have been appropriately recognized and assessed.
5. The project report has been prepared under my supervision, ensuring adherence to high standards of quality, clarity, and structure.

I further certify that this project report has not been previously submitted in part or full for the award of any degree or diploma by any institution or university.

Name of the Guide: Mr. Lalendra Kumar Khanday

Date:

Place: Ibrahimpatnam, Hyderabad.

Name of the HOD: Dr. Mahesh Lokhande

Department: CSE – (AI&ML)

Date:

Place: Ibrahimpatnam, Hyderabad.

ACKNOWLEDGEMENT

"Task successful" makes everyone happy. But the happiness will be gold without glitter if i didn't state the persons who have supported us to make it a success.

I would like to express our sincere thanks to our Guide **Mr. Lalendra Kumar Khanday**, Assistant Professor, Department of Computer Science and Engineering (Artificial Intelligence and Machine Learning), Guru Nanak Institute of Technology for guiding me in developing the requisite capabilities for taking up this project and also for providing seamless support and right suggestions that are given in the development of the project.

I am also thankful to Dr. Mahesh Lokhande, Head of the Department, Computer Science and Engineering (Artificial Intelligence & Machine Learning), for his expert supervision and helpful suggestions, which contributed significantly to the successful completion of this project.

I would also like to thank all our lecturers for helping me in every possible way whenever the need arose.

On a more personal note, I thank our beloved parents and friends for their moral support during our project.

SRAVANTHI KACHAKAYALWAR

23831A66E3

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	CERTIFICATE	i
	PROJECT COMPLETION CERTIFICATE	ii
	DECLARATION BY STUDENTS	iii
	DECLARATION BY GUIDE	iv
	ACKNOWLEDGEMENT	v
	ABSTRACT	ix
	LIST OF FIGURES	x
	LIST OF SYMBOLS	xi-xiii
	ABBREVIATIONS	xiv
1.	INTRODUCTION	1-7
	1.1 GENERAL	1
	1.2 OBJECTIVE	2
	1.3 SCOPE OF THE PROJECT	2
	1.4 EXISTING SYSTEM	2
	1.4.1 EXISTING SYSTEM DISADVANTAGES	2
	1.5 LITERATURE SURVEY	3-6
	1.6 PROPOSED SYSTEM	7
	1.6.1 PROPOSED SYSTEM ADVANTAGES	7
2.	PROJECT DESCRIPTION	8-16
	2.1 GENERAL	8
	2.2 METHODOLOGIES	8-12
	2.2.1 MODULES NAME	8
	2.2.2 MODULES EXPLANATION	9-10
	2.2.3 MODULE DIAGRAM	11-12

	2.3 GIVEN INPUT EXPECTED OUTPUT	13
	2.4 TECHNIQUE OR ALGORITHM	14-16
	2.4.1 PROPOSED ALGORITHM	14-15
	2.4.2 EXISTING SYSTEM	16
3.	REQUIREMENTS	17-20
	3.1 GENERAL	17
	3.2 HARDWARE REQUIREMENTS	17
	3.3 SOFTWARE REQUIREMENTS	17-18
	3.4 FUNCTIONAL SPECIFICATION	18-19
	3.5 NON-FUNCTIONAL SPECIFICATION	19-20
4.	SYSTEM DESIGN	21-34
	4.1 GENERAL	21
	4.2 UML	21-28
	4.2.1 USE CASE DIAGRAM	21-22
	4.2.2 CLASS DIAGRAM	22-23
	4.2.3 OBJECT DIAGRAM	23
	4.2.4 COMPONENT DIAGRAM	23-24
	4.2.5 DEPLOYMENT DIAGRAM	24-25
	4.2.6 SEQUENCED DIAGRAM	25-26
	4.2.7 COLLABARATION DIAGRAM	26
	4.2.8 STATE DIAGRAM	27
	4.2.9 ACTIVITY DIAGRAM	28
	4.3 DATA FLOW DIAGRAM	29-31
	4..4 ER DIAGRAM	32-33
	4.5 SYSTEM ARCHETECTURE	33-34

5.	DEVELOPMENT TOOLS 5.1 GENERAL 5.2 FEATURES OF JAVA 5.2.1 THE JAVA FRAMEWORK 5.2.2 OBJECTIVE OF JAVA 5.2.3 COMPONENTS OF JAVA FRAMEWORK 5.2.4 EVOLUTION OF COLLECTION 5.3 CONCLUSION	35-38 35 35-38 35 35-36 36-37 37-38 38
6.	IMPLEMENTATION 6.1 IMPLEMENTATION	39-42 39-42
7.	SNAPSHOTS 7.1 GENERAL 7.2 VARIOUS SNAPSHOTS	43-44 43 43-44
8.	SOFTWARE TESTING 8.1 GENERAL 8.2 DEVELOPING METHODOLOGIES 8.3 TYPES OF TESTING	45-48 45 45 45-48
9.	CONCLUSION 9.1 CONCLUSION 9.2 FUTURE ENHANCEMENT 9.3 REFERENCES	49-51 49 49 50-51

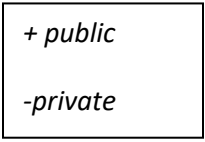
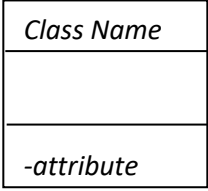
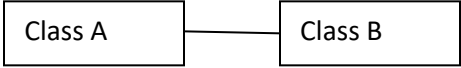
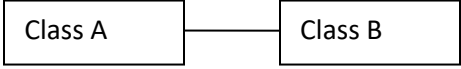
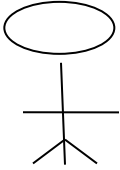
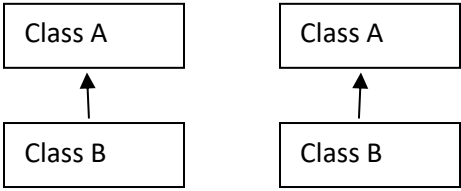
ABSTRACT

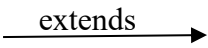
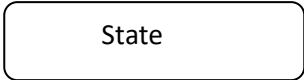
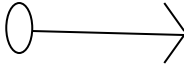
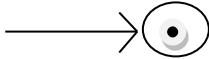
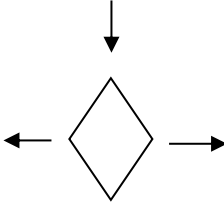
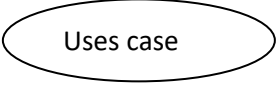
We show how to implement a deniable encryption method from secret sharing. Unlike the related concept of honey encryption, which employs a preprocessing step in symmetric encryption to reshape the distribution of a plaintext towards making the real plaintext indistinguishable from a ciphertext for a fake message, we can avoid both, computational intractability assumptions and preprocessing of the data. This accomplishes deniability against an attacker that can force decryptions, and it can brute-force break a ciphertext with sufficient computational power. Following the concept of plausible deniability, we herein have different decryption keys to open up distinct plaintexts from within the same ciphertext. For instance, a plaintext revealed from a ciphertext with a key which was shared by a victim under duress, will convince the attacker that it is real, while the actual secret remains unnoticed. False Bottom Encryption constructs a symmetric scheme (in the sense of using the same key to encrypt and decrypt) that shares the properties of both honey encryption and deniable encryption. We specifically formalize and differentiate “deniable” from “plausibly deniable” as a security feature, showing how plausible deniability falls back to (only) deniability, depending on the plaintext distribution. Our scheme is simple, lightweight to implement and efficient in terms of encryption and decryption, and is based on secret sharing.

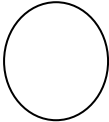
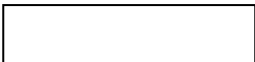
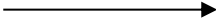
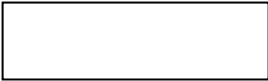
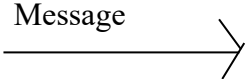
LIST OF FIGURES

FIGURE NO	NAME OF THE FIGURE	PAGE NO.
2.3.2	Module Diagram	9-12
4.2.1	Use case Diagram	21
4.2.2	Class Diagram	22
4.2.3	Object Diagram	23
4.2.4	Component Diagram	23
4.2.5	Deployment Diagram	24
4.2.6	Sequence Diagram	25
4.2.7	Collaboration Diagram	26
4.2.8	State Diagram	27
4.2.9	Activity Diagram	28
4.3	Data Flow Diagram	29-31
4.4	E-R Diagram	32
4.5	System Architecture	33
7.2.1	Owners Registration	43
7.2.2	Users Registration	43
7.2.3	Approving Key Requests	44
7.2.4	Decrypted Data	44

LIST OF SYMBOLS

S.NO	NAME	NOTATION	DESCRIPTION
1.	Class	 	Represents a collection of similar entities grouped together.
2.	Association	 	Associations represents static relationships between classes. Roles represents the way the two classes see each other.
3.	Actor		It aggregates several classes into a single classes.
4.	Aggregation		Interaction between the system and external environment.
5.	<i>Relation</i> (uses)	<i>Uses</i>	Used for additional process communication.

6.	Relation (extends)		Extends relationship is used when one use case is similar to another use case but does a bit more.
7.	Communication		Communication between various use cases.
8.	State		State of the process.
9.	Initial State		Initial state of the object
10.	Final state		Final state of the object
11.	Control flow		Represents various control flow between the states.
12.	Decision box		Represents decision making process from a constraint
13.	Usecase		Interaction between the system and external environment.

14.	Component		Represents physical modules which is a collection of components.
15.	Node		Represents physical modules which are a collection of components.
16.	Data Process/State		A circle in DFD represents a state or process which has been triggered due to some event or action.
17.	External entity		Represents external entities such as keyboard,sensors,etc.
18.	Transition		Represents communication that occurs between processes.
19.	Object Lifeline		Represents the vertical dimensions that the object communications.
20.	Message		Represents the message exchanged.

LIST OF ABBREVIATION

S.NO	ABBREVIATION	EXPANSION
1.	DB	DataBase
2.	JVM	Java Virtual Machine
3.	JSP	Java Server Page
4.	PWS	Personalised Web Search
5.	UPS	User Personalised Search
6.	JRE	Java Runtime Environment

Denial of Encryption From Secret Sharing in Data

CHAPTER 1

INTRODUCTION

1.1 GENERAL

Generally speaking, a sender and receiver must first share information to communicate safely. In many cases, encryption and good password protection may be sufficient to safeguard your data. For instance, with AES, the sender and receiver share the same key in a symmetric encryption system. However, using the RSA technique, the sender and receiver of an asymmetric-key encryption technique exchange a public system parameter and the recipient's public key, with the public key delivery occurring through public key infrastructure. These general encryption techniques offer a security guarantee against eavesdropping attempts, but they fall short when faced with threats of coercion. Even if the attacker does not have access to the key, if it intercepts the ciphertext, it may be able to force both the sender and the receiver to decrypt the message. Non-committing encryption [1] and deniable encryption [2] have been presented as solutions to this issue. Users can decrypt an existing ciphertext associated with a certain counterfeit message using these two different encryption algorithms. The first algorithm is called the sender's encryption algorithm to encrypt a message under a secret key sk . The second algorithm, known as the faking algorithm, is publicly known, and the sender uses this fake algorithm to produce fake messages. Getting the same ciphertext from two different algorithms is computationally cumbersome. In our work, we show how the actual message and the fake message ciphertexts can both be produced using a single algorithm that also is computationally efficient. There are many circumstances under which plausible deniability may also be necessary. If your opponents cannot obtain your password, strong encryption can keep them out. However, if the threat model incorporates coercion, such as the prospect of a jail term or torture, you might give up and hand over the key to rescue yourself. As a result, the attacker would have access to the data, perhaps putting you at risk for later repercussions. The idea of plausible deniability originates in politics and espionage and refers to one's capacity to downplay one's culpability for, or knowledge of specific facts or events.

1.2 OBJECTIVE

Generally, a sender and receiver must first share information to communicate safely. In many cases, encryption and good password protection may be sufficient to safeguard your data. For instance, with AES, the sender and receiver share the same key in a symmetric encryption system. the public key, with the public key delivery occurring through public key infrastructure. cipher text, it may be able to both the sender and the receiver to decrypt the data stored.

1.3 SCOPE OF THE PROJECT

The scope of the project the construction of honey encryption makes use of distribution-transforming encoders that aim to shape the distribution of a random plaintext towards a desired and fixed target distribution. Our scheme can use such encoders as well, as a source of plausibly looking plaintexts to act as fakes. We will not make explicit use of such transformations, but mention them as a possible technical implementation of our assumption that fake plaintexts are producible with the same distribution as the real secret plaintexts.

1.4 EXISTING SYSTEM:

- Users can decrypt an existing cipher text associated with a certain counterfeit message using these two different encryption algorithms.
- The first algorithm is called the sender's encryption algorithm to encrypt a message under a secret key sk .
- The second algorithm, known as the faking algorithm, is publicly known, and the sender uses this fake algorithm to produce fake messages. Getting the same cipher text from two different algorithms is computationally cumbersome.

1.4.1 Existing System Disadvantages:

- No Security authentication.
- Not detect a attackers.
- No Access data anybody can attacks a files.

1.5 LITERATURE SURVEY

TITLE : Hierarchical attribute based encryption with continuous leakage-resilience

AUTHOR : J. Li, Q. Yu, and Y. Zhang, ‘

YEAR :2023

DESCRIPTION :

Attribute based encryption (ABE) is widely applied in cloud computing settings due to its fine-grained access control. Most ABE schemes do not consider the side channel attacks which may leak the secret information of cryptosystems. Leakage-resilient cryptography aims to model security for various side channel attacks. In this paper, we first give the formal definition and security model of hierarchical attribute based encryption (HABE) with continuous leakage-resilience. Furthermore, we present a ciphertext-policy HABE scheme with continuous leakage-resilience. The proposed scheme is resilient to master key leakage and secret key leakage. We prove the security of our scheme under composite order bilinear group assumptions by using dual system encryption techniques. The performance of leakage-resilience is analyzed theoretically. If the depth of our proposed scheme is 1, the relative leakage ratio is almost up to $1/3$. In addition, we give the performance comparison through experiments.

TITLE : ‘PD-DM: An efficient locality-preserving block device mapper with plausible deniability

AUTHOR : C. Chen, A. Chakraborti, and R. Sion,

YEAR : 2023

DESCRIPTION :

Encryption protects sensitive data from unauthorized access, yet is not sufficient when users are forced to surrender keys under duress. In contrast, plausible deniability enables users to not only encrypt data but also deny its existence when challenged. Most existing plausible deniability work (e.g. the successful and unfortunately now-defunct TrueCrypt) tackles “single snapshot” adversaries, and cannot handle the more realistic scenario of adversaries gaining access to a device at multiple time points. Such “multi-snapshot” adversaries can simply observe modifications between snapshots and detect the existence of hidden data. Existing ideas handling “multi-

snapshot” scenarios feature prohibitive overheads when deployed on practically-sized disks. This is mostly due to a lack of data locality inherent in certain standard access randomization mechanisms, one of the building blocks used to ensure plausible deniability. In this work, we show that such randomization is not necessary for strong plausible deniability.

TITLE : SoK: Plausibly deniable storage,’

AUTHOR : Hakan Ancin,Xi Chen

YEAR : 2022

DESCRIPTION :

Data privacy is critical in instilling trust and empowering the societal pacts of modern technology-driven democracies. Unfortunately, it is under continuous attack by overreaching or outright oppressive governments, including some of the world’s oldest democracies. Increasingly-intrusive anti-encryption laws severely limit the ability of standard encryption to protect privacy. New defense mechanisms are needed. Plausible deniability (PD) is a powerful property, enabling users to hide the existence of sensitive information in a system under direct inspection by adversaries. Popular encrypted storage systems such as TrueCrypt and other research efforts have attempted to also provide plausible deniability. Unfortunately, these efforts have often operated under less well-defined assumptions and adversarial models.

It aims to:

1. identify key PD properties, requirements, and approaches;
2. present a direly-needed unified framework for evaluating security and performance;
3. explore the challenges arising from the critical interplay between PD and modern system layered stacks.
4. propose a new “trace-oriented” PD paradigm, able to decouple security guarantees from the underlying systems and thus ensure a higher level of flexibility and security independent of the technology stack.

This work is meant also as a trusted guide for system and security practitioners around the major challenges in understanding, designing, and implementing plausible deniability into new or existing systems.

TITLE : ‘Audit-free cloud storage via deniable attribute based encryption

AUTHOR : P. Chi and C. Lei

YEAR : 2022

DESCRIPTION :

Cloud storage services have become increasingly popular. Because of the importance of privacy, many cloud storage encryption schemes have been proposed to protect data from those who do not have access. All such schemes assumed that cloud storage providers are safe and cannot be hacked; however, in practice, some authorities (i.e., coercers) may force cloud storage providers to reveal user secrets or confidential data on the cloud, thus altogether circumventing storage encryption schemes. In this paper, we present our design for a new cloud storage encryption scheme that enables cloud storage providers to create convincing fake user secrets to protect user privacy. Since coercers cannot tell if obtained secrets are true or not, the cloud storage providers ensure that user privacy is still securely protected.

TITLE : Full verifiability for outsourced decryption in attribute based encryption.

AUTHOR : . Li, Y. Wang, Y. Zhang, and J. Han,

YEAR : 2023

DESCRIPTION :

Attribute based encryption (ABE) is a popular cryptographic technology to protect the security of users' data. However, the decryption cost and cipher text size restrict the application of ABE in practice. For most existing ABE schemes, the decryption cost and cipher text size grow linearly with the complexity of access structure. This is undesirable to the devices with limited computing capability and storage space. Outsourced decryption is considered as a feasible method to reduce the user's decryption overhead, which enables a user to outsource a large number of decryption operations to the cloud service provider (CSP). However, outsourced decryption cannot guarantee the correctness of transformation done by the cloud, so it is necessary to check the correctness of outsourced decryption to ensure security for users' data. Current research mainly focuses on verifiability of outsourced decryption for the authorized users. It still remains a challenging issue

that how to guarantee the correctness of outsourced decryption for unauthorized users. In this paper, we propose an ABE scheme with verifiable outsourced decryption (called full verifiability for outsourced decryption), which can simultaneously check the correctness for transformed ciphertext for the authorized users and unauthorized users. The proposed ABE scheme with verifiable outsourced decryption is proved to be selective CPA-secure in the standard model.

TITLE : Verifiable symmetric searchable encryption for semi-honest-but-curious cloud servers

AUTHOR : G. Gong and Q. Cha

YEAR : 2024

DESCRIPTION :

Outsourcing data to cloud servers, while increasing service availability and reducing users' burden of managing data, inevitably brings in new concerns such as data privacy, since the server may be honest-but-curious. To mediate the conflicts between data usability and data privacy in such a scenario, research of searchable encryption is of increasing interest. Motivated by the fact that a cloud server, besides its curiosity, may be selfish in order to save its computation and/or download bandwidth, in this paper, we investigate the searchable encryption problem in the presence of a semi-honest-but-curious server, which may execute only a fraction of search operations honestly and return a fraction of search outcome honestly. To fight against this strongest adversary ever, a verifiable SSE (VSSE) scheme is proposed to offer verifiable searchability in addition to the data privacy, both of which are further confirmed by our rigorous security analysis. Besides, we treat the practicality/efficiency as a central requirement of a searchable encryption scheme. To demonstrate the lightweightness of our scheme, we implemented and tested the proposed VSSE on a laptop (serving as the server) and a mobile phone running Android 2.3.4 (serving as the end user). The experimental results optimistically suggest that the proposed scheme satisfies all of our design goals.

1.6 PROPOSED SYSTEM

- Unlike the related concept of honey encryption, which employs a preprocessing step in symmetric encryption to re-shape the distribution of a plaintext towards making the real plaintext indistinguishable from a cipher text data.
- For instance, with AES, the sender and receiver share the same key in a symmetric encryption system.
- A deniable shared key scheme and a public key scheme are two types of deniable encryption. A straight forward illustration of deniable encryption is the one-time pad: Let m be the original message to be encrypted data.
- In the area of symmetric encryption, ambiguous multi-symmetric cryptography has been proposed with goals similar to ours. They provide a discussion of various attack scenarios, including known-plaintext, chosen cipher text and others

1.6.1 PROPOSED SYSTEM ADVANTAGE

- Stronger authentication
- The user can detect whether the server is launching an attack.
- the attacker will recognize the correct message from a list of possible plaintexts

CHAPTER 2

PROJECT DESCRIPTION

2.1 GENERAL:

In the area of symmetric encryption, ambiguous multi-symmetric cryptography has been proposed with goals similar to ours. They provide a discussion of various attack scenarios, including known-plaintext, chosen cipher text and others, all boiling down to the argument that this leaves the adversary with a linear system of equations that is under-determined. In basing their construction on number theoretic arguments, they nonetheless rely on computational intractability (of finding factorizations or primes, though not via a usual reductionist argument). The computational complexity of this prior construction is governed by Chinese remaindering. Our scheme improves over this in requiring only linear efforts (a number of field operations that is proportional to the number of inner plaintexts).

2.2 METHODOLOGIES

2.2.1 MODULES NAME:

1. User Interface Design
2. Authority Server
3. Owner
4. User
- 5.CSP

1. User Interface Design

In this module we design the windows for the project. These windows are used for secure login for all users. To connect with server user must give their username and password then only they can able to connect the server. If the user already exists directly can login into the server else user must register their details such as username, password and Email id, into the server. Server will create the account for the entire user to maintain upload and download rate. Name will be set as user id. Logging in is usually used to enter a specific page.

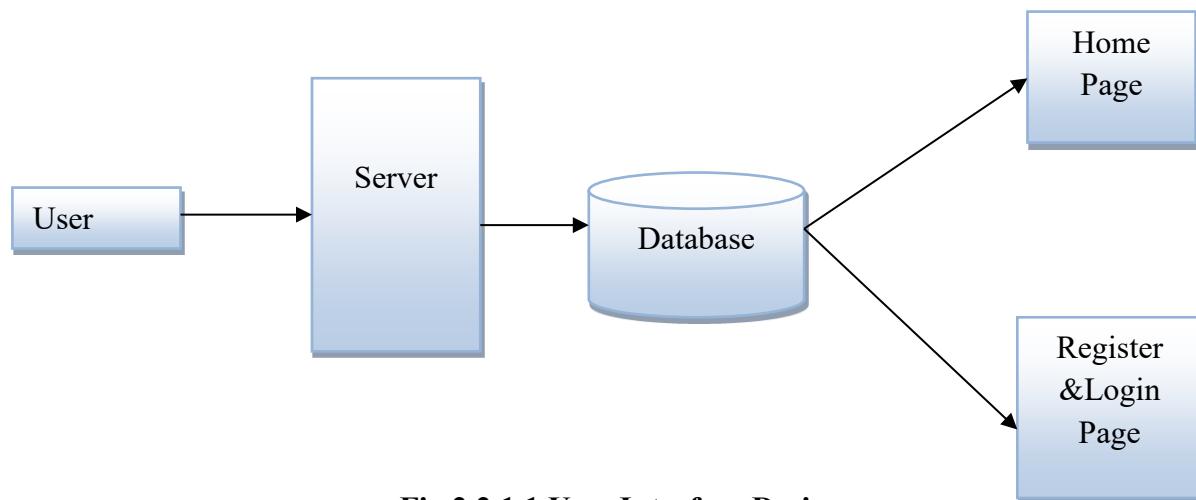


Fig.2.2.1.1 User Interface Design

2. Authority Server

The first module is a authenticator server. First server has to login with a user id and password. Authority server has a key request. The authority server has a generate a key generator. The authority server has a requests. It will have a owner requests and it was also have a user requests from the database.

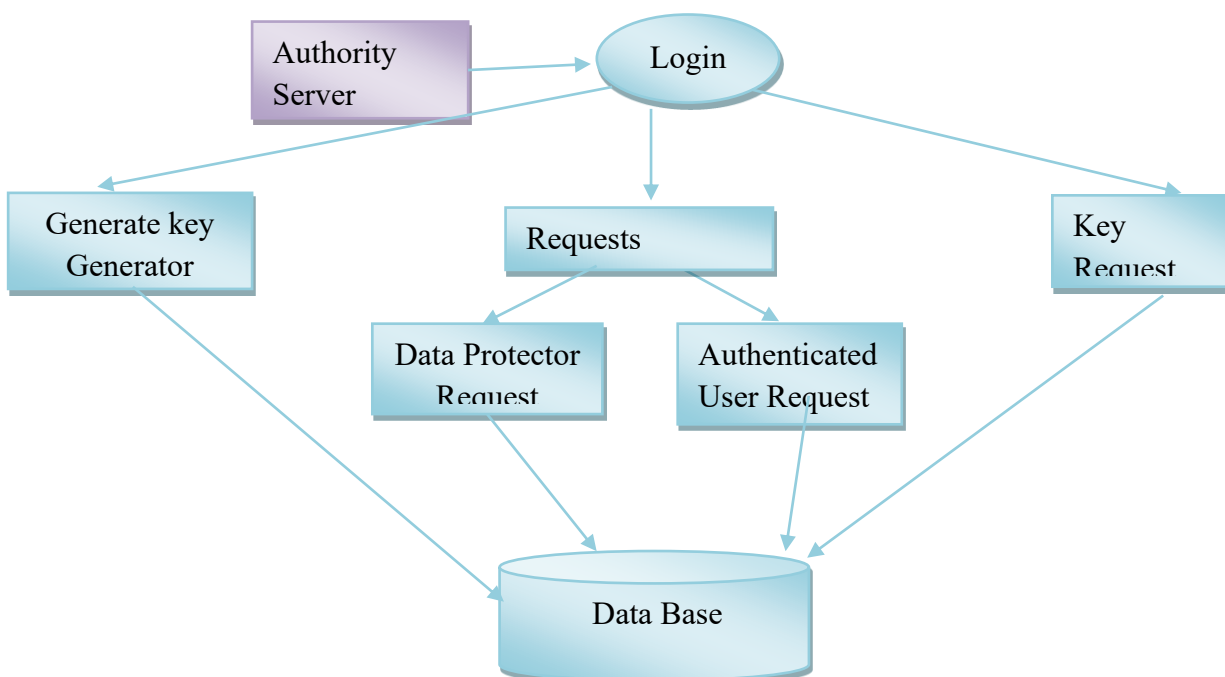


Fig.2.2.1.2 Authority Server

3. Owner

The third module. Owner has a register and login takes permission from the authority server. Authority server has a login with a user id and password. Owner has a uploaded data store. We can view a uploaded view data. Owner has a approve a data to share.

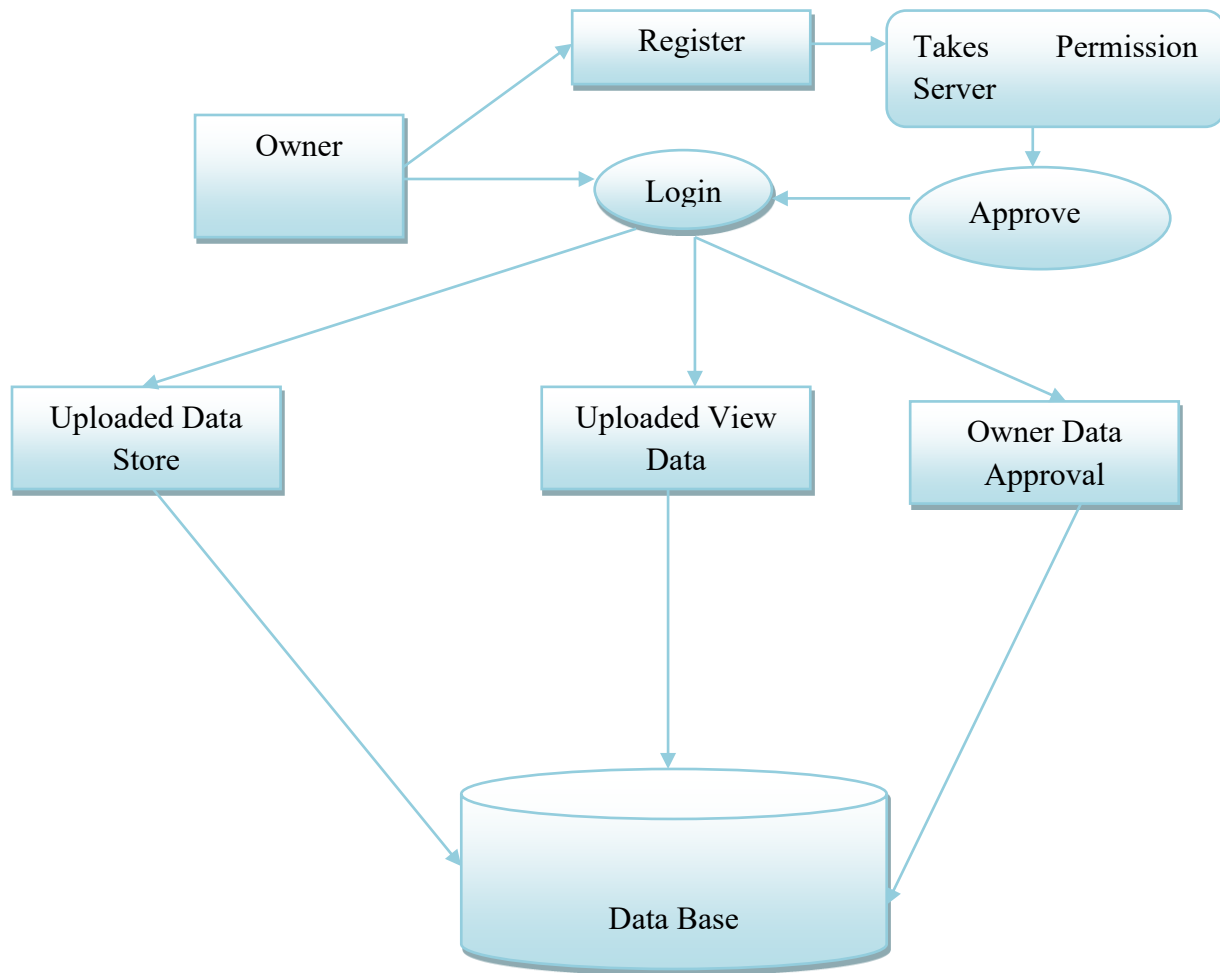


Fig.2.2.1.3 Owner

4. User

The fourth module is a user. User has a register with a user id and password. User has a to login takes permissions from the authority center. Authority server has to add a user then login. User has a data search a data.

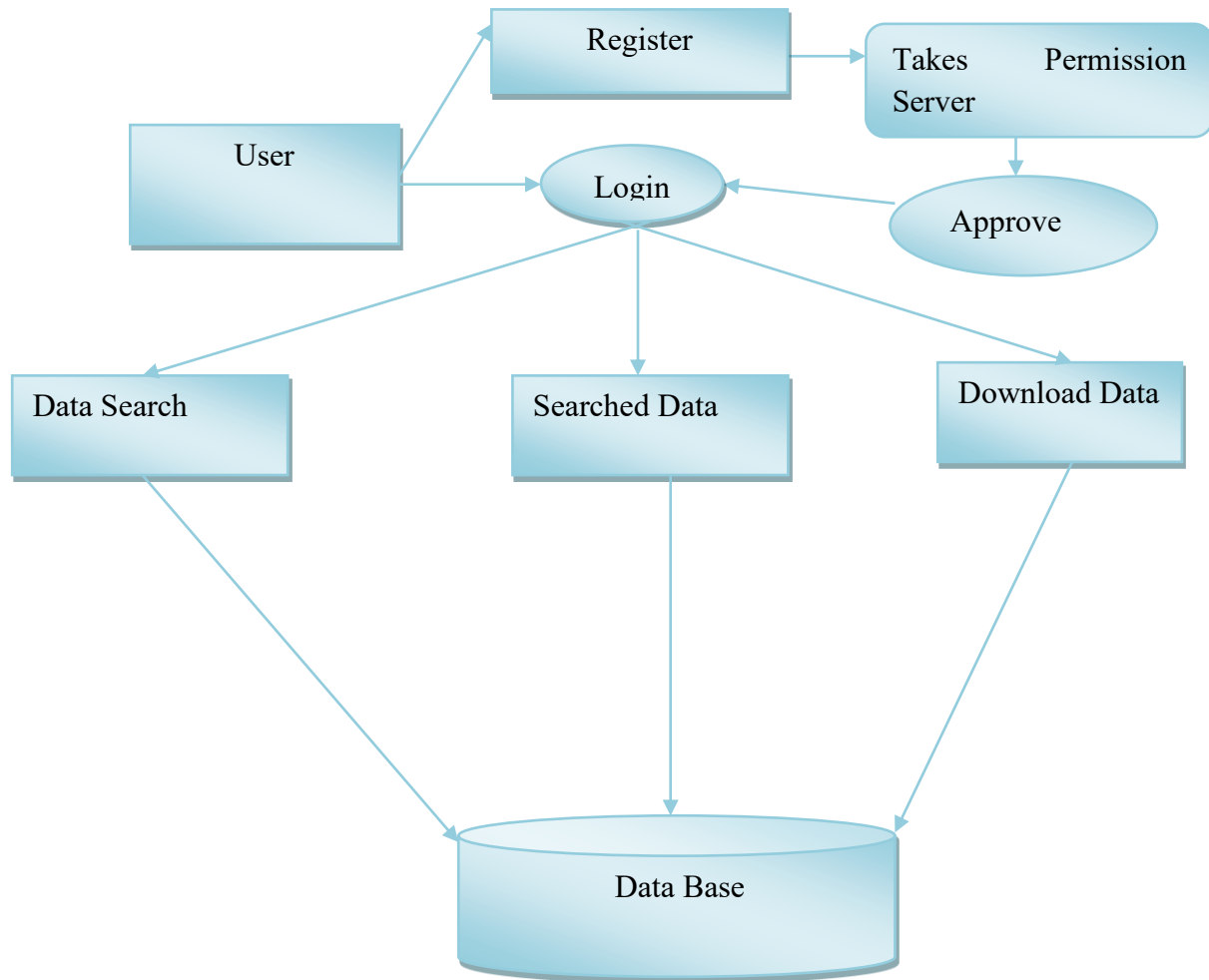


Fig.2.2.1.4 User

5. CSP(Cloud Service Provider)

Cloud has a fifth module. CSP has a login with a user id and password. CSP has a Stored data in the database. CSP has a details to have a owner details and it have a user details. The stores in a database.

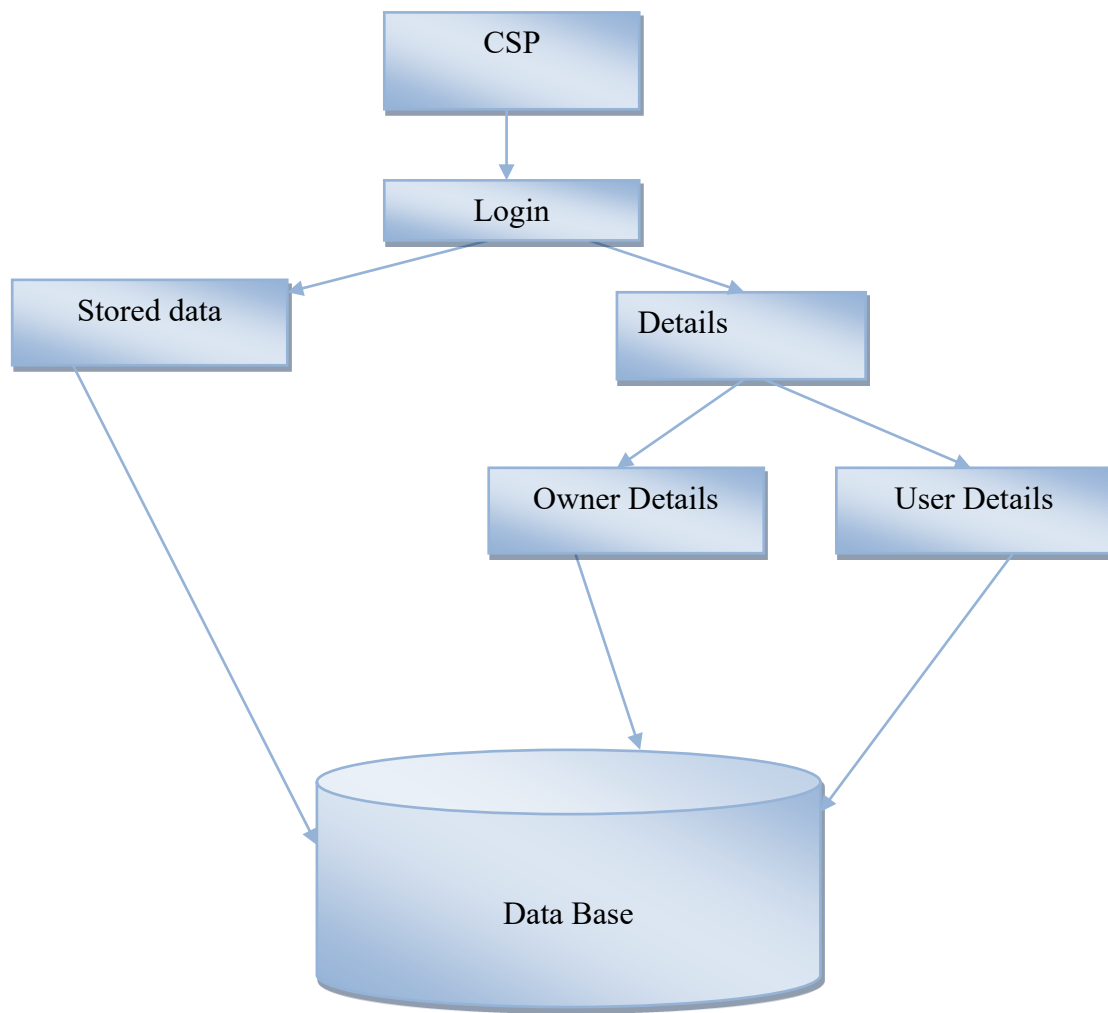


Fig.2.2.1.5 CSP

2.3 GIVEN INPUT EXPECTED OUTPUT:

➤ User Interface Design

Input : Enter Login name and Password

Output : If valid user name and password then directly open the home page otherwise show error message and redirect to the registration page.

➤ Authority Server

Input : AS have a Login name and Password

Output: If valid user name and password then directly open the authority server home page otherwise show error message and redirect to the authority server login page.

➤ Owner

Input : Enter the owner mail id and password

Output : If valid manager id and password then directly open the owner home page otherwise show error message and redirect to the owner login page.

➤ User

Input : Enter the user email and password

Output: If valid Authenticated user id and password then directly open the iu home page otherwise show error message and redirect to the user login page.

➤ CSP

Input : Enter the CSP name and password

Output: If valid CSP name and password then directly open the CSP home page otherwise show error message and redirect to the CSP login page.

2.4 TECHNIQUE USED OR ALGORITHM USED

2.4.1 Proposed Algorithm

1.Honey Encryption

A Honey Encryption involves repeated decryption with random keys; this is equivalent to picking random plaintexts from the space of all possible plaintexts with a uniform distribution. This is effective because even though the attacker is equally likely to see any given plaintext, most plaintexts are extremely unlikely to be legitimate i.e. the distribution of legitimate plaintexts is *non*-uniform. Honey encryption defeats such attacks by first transforming the plaintext into a space such that the distribution of legitimate plaintexts *is* uniform. Thus an attacker guessing keys will see legitimate-looking plaintexts frequently and random-looking plaintexts infrequently. This makes it difficult to determine when the correct key has been guessed. In effect, honey encryption "[serves] up fake data in response to every incorrect guess of the data or encryption key.

The security of honey encryption relies on the fact that the probability of an attacker a plaintext to be legitimate can be calculated (by the encrypting party) at the encryption. This makes honey encryption difficult to apply in certain applications e.g. where the space of plaintexts is very large or the distribution of plaintexts is unknown. It also means that honey encryption can be vulnerable if this probability is miscalculated. For example, it is vulnerable to known-plaintext attacks: if the attacker has a crib that a plaintext must match to be legitimate, they will be able to brute-force even Honey Encrypted data if the encryption did not take the crib into account.

2.Hash Algorithm

There are majorly three components of hashing:

1. Key: A Key can be anything string or integer which is fed as input in the hash function the technique that determines an index or location for storage of an item in a data structure.
2. Hash Function: The hash function receives the input key and returns the index of an element in an array called a hash table. The index is known as the hash index.
3. Hash Table: Hash table is a data structure that keys to values using a special function called a hash function. Hash stores the data in an associative manner in an array where each data value has its own unique index.

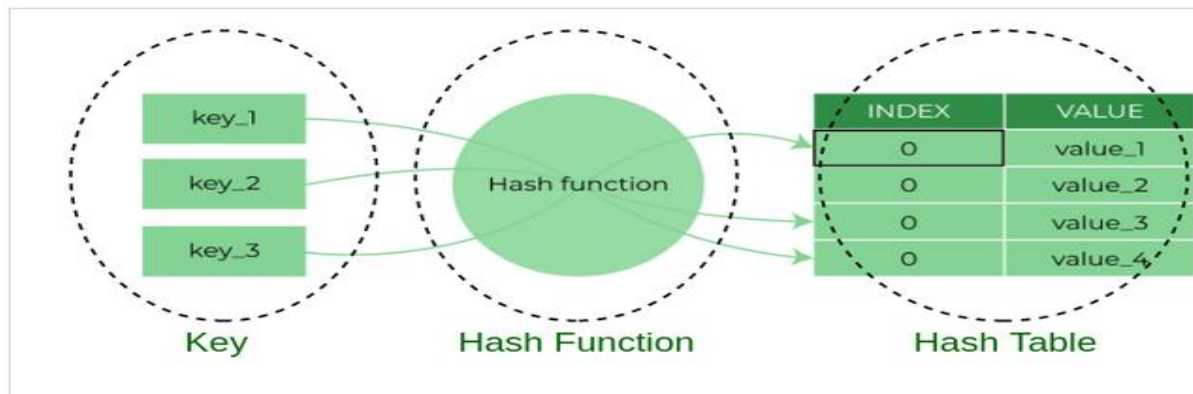


Figure 2.4.1 Hash algorithm Example

Real-Time Applications of Hash Data structure

- Hash is used for cache data for fast access to the data.
- Hash can be used for password verification.
- Hash is used in cryptography as a message digest.

3. SYMMETRIC ALGORITHM USED

Symmetric-key algorithms are algorithms for cryptography that use the same cryptographic keys for both the encryption of plaintext and the decryption of cipher text. The keys may be identical, or there may be a simple transformation to go between the two keys. The keys, in practice, represent a shared secret between two or more parties that can be used to maintain a private information link. However, symmetric-key encryption algorithms are usually better for bulk encryption. With exception of the one-time pad they have a smaller key size, which means less storage space and faster transmission. Due to this, asymmetric-key encryption is often used to exchange the secret key for symmetric-key encryption.

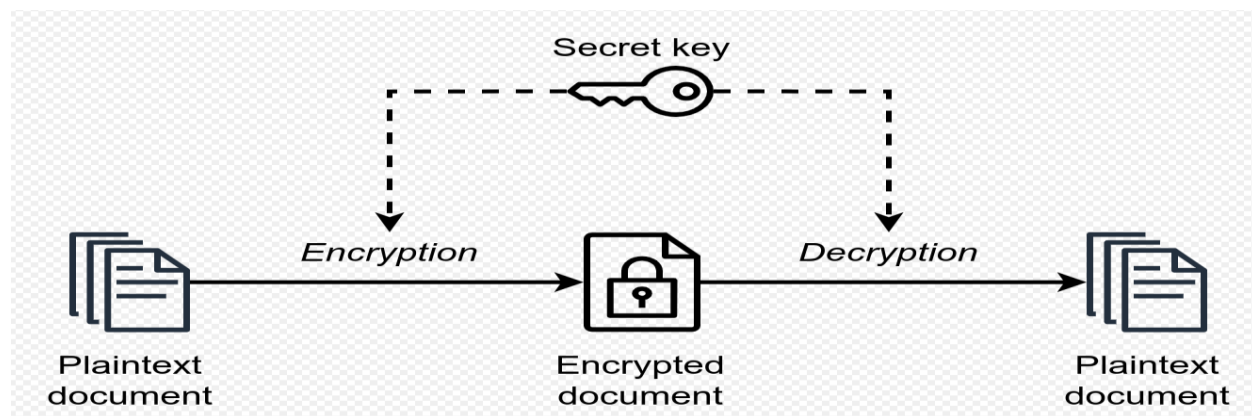


Figure 2.4.2 Symmetric key algorithm

2.4.2 EXISTING SYSTEM

Attribute-based encryption (ABE) is a type of broadcast encryption

Attribute-based encryption is a generalization of public-key encryption which enables fine grained access control of encrypted data using authorization policies. The secret key of a user and the cipher text are dependent upon attributes (e.g. their email address, the country in which they live, or the kind of subscription they have). In such a system, the decryption of a cipher text is possible only if the set of attributes of the user key matches the attributes of the cipher text.^[1]

A crucial security aspect of attribute-based encryption is collusion-resistance: An adversary that holds multiple keys should only be able to access data if at least one individual key grants access. Attribute-based encryption (ABE) is a relatively recent approach that reconsiders the concept of public-key cryptography. In traditional public-key cryptography, a message is encrypted for a specific receiver using the receiver's public-key. Identity-based cryptography and in particular identity-based encryption (IBE) changed the traditional understanding of public-key cryptography by allowing the public-key to be an arbitrary string, e.g., the email address of the receiver. ABE goes one step further and defines the identity not atomic but as a set of attributes, e.g., roles, and messages can be encrypted with respect to subsets of attributes (key-policy ABE - KP-ABE) or policies defined over a set of attributes (ciphertext-policy ABE - CP-ABE). The key issue is, that someone should only be able to decrypt a ciphertext if the person holds a key for "matching attributes" (more below) where user keys are always issued by some trusted party.

CHAPTER 3

REQUIREMENTS ENGINEERING

3.1 GENERAL

We have conducted experiments on our collected dataset and extensive results have demonstrated that our model outperforms all other existing models. In the future, we will investigate more tasks under this framework, such as event summarization and event attribute mining in social media.

3.2 HARDWARE REQUIREMENTS

The hardware requirements may serve as the basis for a contract for the implementation of the system and should therefore be a complete and consistent specification of the whole system. They are used by software engineers as the starting point for the system design. It should what the system do and not how it should be implemented.

- PROCESSOR : DUAL CORE 2 DUOS.
- RAM : 2GB DD RAM
- HARD DISK : 250 GB

3.3 SOFTWARE REQUIREMENTS

The software requirements document is the specification of the system. It should include both a definition and a specification of requirements. It is a set of what the system should do rather than how it should do it. The software requirements provide a basis for creating the software requirements specification. It is useful in estimating cost, planning team activities, performing tasks and tracking the teams and tracking the team's progress throughout the development activity.

SOFTWARE REQUIREMENTS

- FRONT END : J2EE (JSP, SERVLET)
- BACK END : MY SQL 5.5
- OPERATING SYSTEM : WINDOWS 7
- IDE : ECLIPSE

3.4 FUNCTIONAL REQUIREMENTS

Authenticator Server (Module 1):

- **Login:** The server should accept a user ID and password for authentication.
- **Key Request:** The server should handle requests from the authority server to generate keys.
- **Key Generation:** The authority server should generate a key based on the user's request.

Authority Server (Module 2):

- **Key Generation:** The authority server should generate a key when requested.
- **Owner Requests:** The authority server should handle requests from the owner, such as registering data for approval.
- **User Requests:** The server should validate requests from users via the database to ensure proper permissions.
- **Owner Permissions:** The owner should be able to register, log in, and grant permissions to access the data.
- **Data Approval:** The owner should approve or deny access to data shared with others.
- **User Permissions:** The authority server should ensure users are authorized and manage their access.

Owner Module (Module 3):

- **Registration/Login:** The owner must register and log in with a user ID and password.
- **Permission from Authority:** The owner must take permission from the authority server for certain actions (such as data sharing).
- **Upload Data Store:** The owner can upload data to the system.
- **Data Approval:** The owner can approve or deny the sharing of uploaded data.

User Module (Module 4):

- **Registration/Login:** The user must register and log in using a user ID and password.
- **Permissions from Authority:** The user must get permissions from the authority server to access the data.
- **Data Search:** The user can search for data stored in the cloud.
- **Data Access:** The user can access data if the owner has approved access and the user has the required permissions.

Cloud Storage Provider (CSP) Module (Module 5):

- **Login:** The CSP should allow login with a user ID and password.
- **Store Data:** The CSP should store the data provided by owners in a secure manner.
- **Store User and Owner Details:** The CSP should maintain records of both owner and user details.
- **Data Access Control:** The CSP should ensure that the data is shared correctly according to the permissions set by the owner and the authority server.

3.5 NON-FUNCTIONAL REQUIREMENTS

EFFICIENCY

Security:

- All modules must ensure data is stored securely and accessed only by authorized parties.
- Passwords and sensitive data should be encrypted.
- Key management should follow industry best practices (e.g., using RSA, AES).
- Two-factor authentication (2FA) could be implemented for extra security.

Scalability:

- The system should handle increasing numbers of users, owners, and data requests without significant degradation in performance.
- Cloud storage and data access systems should be able to scale horizontally as data grows.

Availability:

- High availability for authentication servers and authority servers.
- Redundancy in database and cloud storage systems to avoid single points of failure.
- The system should support backup and recovery mechanisms for critical data.

Performance:

- Fast login and data retrieval processes, minimizing latency.
- Key generation and validation should be optimized for speed.
- Data search and upload actions should be efficient.

Reliability:

- The system must be reliable with minimal downtime.
- User, owner, and data details should be consistently accurate and up-to-date.

Usability:

- The system should provide an easy-to-use interface for owners, users, and administrators.
- Error messages should be clear, and users should be guided through any issues (e.g., forgotten passwords).

Interoperability:

- The modules should be able to communicate seamlessly (e.g., API integration between the authority server, owner, user, and CSP).
- The system should be able to integrate with existing infrastructure, databases, and authentication protocols.

Compliance:

- The system should comply with data protection regulations (e.g., GDPR, HIPAA) depending on the geographical location.
- It should support audit logs for tracking access and changes to data.

Maintainability:

- The system should be modular to allow easy updates or modifications.
- Clear and documented codebase for easier troubleshooting and updates.
- Logs and monitoring systems should be in place to track performance and errors.

CHAPTER 4

DESIGN ENGINEERING

4.1 GENERAL

Design Engineering deals with the various UML [Unified Modeling language] diagrams for the implementation of project. Design is a meaningful engineering representation of a thing that is to be built. Software design is a process through which the requirements are translated into representation of the software. Design is the place where quality is rendered in software engineering. Design is the means to accurately translate customer requirements into finished product.

4.2 UML DIAGRAMS

4.2.1 USE CASE DIAGRAM

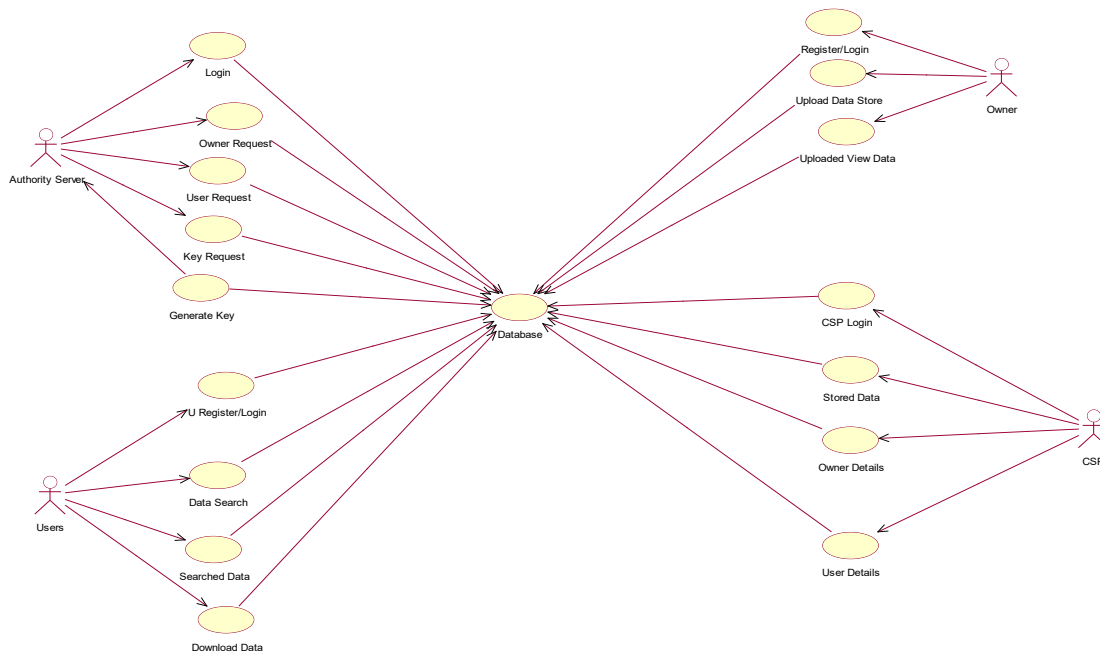


Fig.4.2.1.1 Usecase Diagram

EXPLANATION:

The main purpose of a use case diagram The Four modules are available. Each detail is recorded in the owner's registration. It requested permission from the authority server to log in. Prior to

logging in, the authority server must approve. Owner responds upon permission, and then it was time to log in. Owner has uploaded a text document as the data to be stored. Owner uploaded their own data. Owner views the data. Owner data is approved by the owner. A user's register contains information. The authority server also grants permissions to the user. The authority server has the ability to approve user requests. The database has a user-uploaded data search. Data was searched by the user. Additionally, users can download data. The login for CSP requires a user ID and password. CSP has data that is stored. The owner and user details are collected by CSP. A user ID and password are required to log into the authority server. A key request was made to the authority server using a key generator. Owner and user requests are on the authority server.

4.2.2 CLASS DIAGRAM

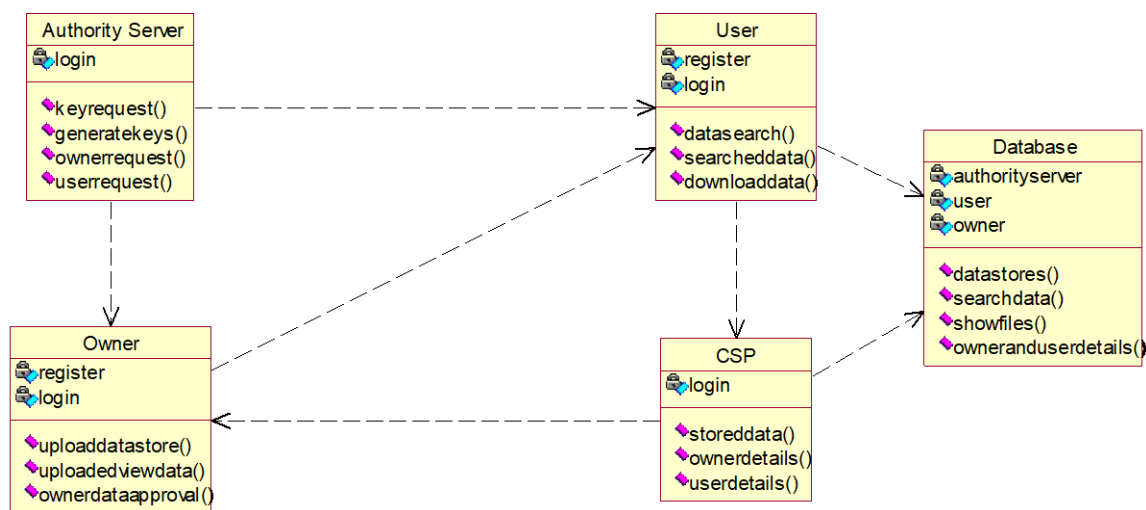


Fig.4.2.2.1 Class Diagram

EXPLANATION

In this class diagram represents The Four modules are available. Each detail is recorded in the owner's registration. It requested permission from the authority server to log in. Prior to logging in, the authority server must approve. Owner responds upon permission, and then it was time to log in. Owner has uploaded a text document as the data to be stored. Owner uploaded their own data. Owner views the data. Owner data is approved by the owner. A user's register contains information. The authority server also grants permissions to the user. The authority server has the ability to approve user requests. The database has a user-uploaded data search. Data was searched by the user. Additionally, users can download data. The login for CSP requires a user ID and password. CSP has data that is stored. The owner and user details are collected by CSP. A user ID and password are required to log into the authority server. A key request was made to the authority server using a key generator. Owner and user requests are on the authority server.

password.CSP has data that is stored. The owner and user details are collected by CSP. A user ID and password are required to log into the authority server. A key request was made to the authority server using a key generator. Owner and user requests are on the authority server.

4.2.3 OBJECT DIAGRAM

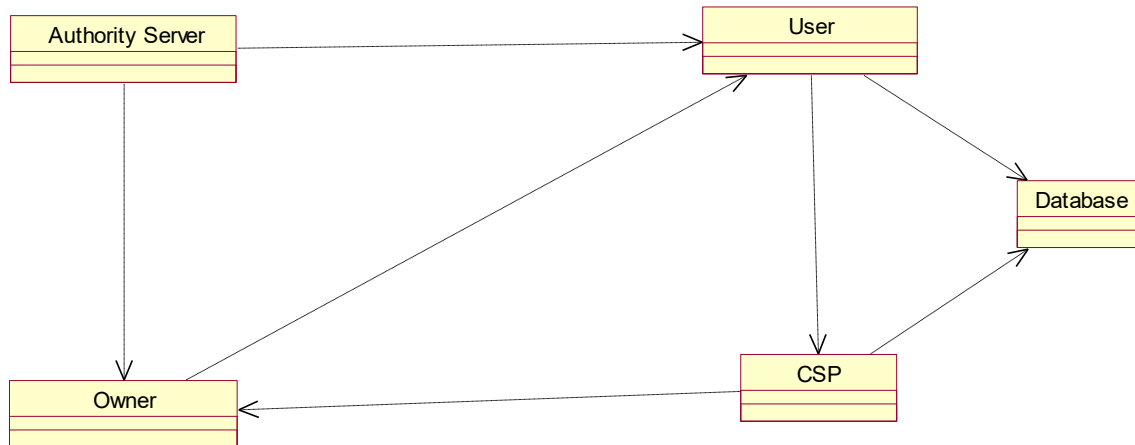


Fig.4.2.3.1 Object Diagram

EXPLANATION:

The authority server is linked to one another in the object diagram. User and owner are connected. The database contains the cloud-stored data.

4.2.4 COMPONENT DIAGRAM

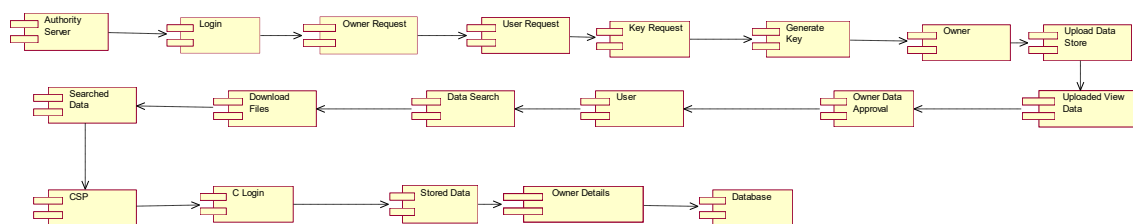


Fig.4.2.4.1 Component Diagram

EXPLANATION

In the Unified Modeling Language, a component diagram The Four modules are available. Each detail is recorded in the owner's registration. It requested permission from the authority server to log in. Prior to logging in, the authority server must approve. Owner responds upon permission, and then it was time to log in. Owner has uploaded a text document as the data to be stored. Owner uploaded their own data. Owner views the data. Owner data is approved by the owner. A user's register contains information. The authority server also grants permissions to the user. The authority server has the ability to approve user requests. The database has a user-uploaded data search. Data was searched by the user. Additionally, users can download data. The login for CSP requires a user ID and password.CSP has data that is stored. The owner and user details are collected by CSP. A user ID and password are required to log into the authority server. A key request was made to the authority server using a key generator. Owner and user requests are on the authority server.. All boxes are components and arrow indicates dependencies.

4.2.5 DEPLOYMENT DIAGRAM

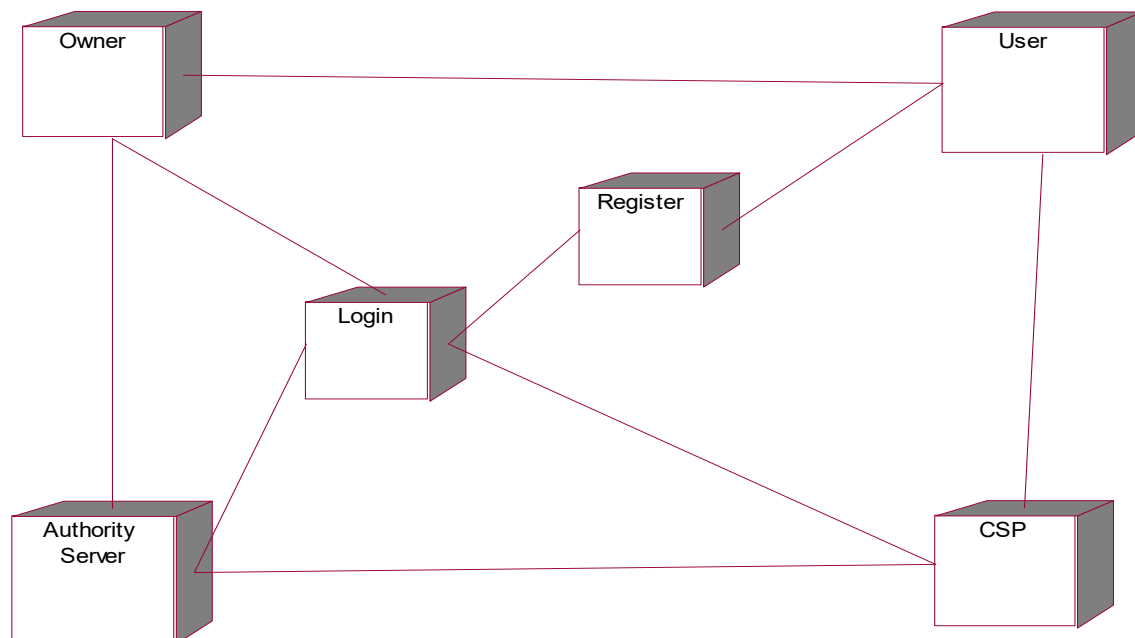


Fig.4.2.5.1 Deployment Diagram

EXPLANATION:

Deployment Diagram is a type of diagram that specifies that a owner has a login. User(User) has a register and then login. Cloud has a login. It was also a stores a database stores.

4.2.6 SEQUENCE DIAGRAM

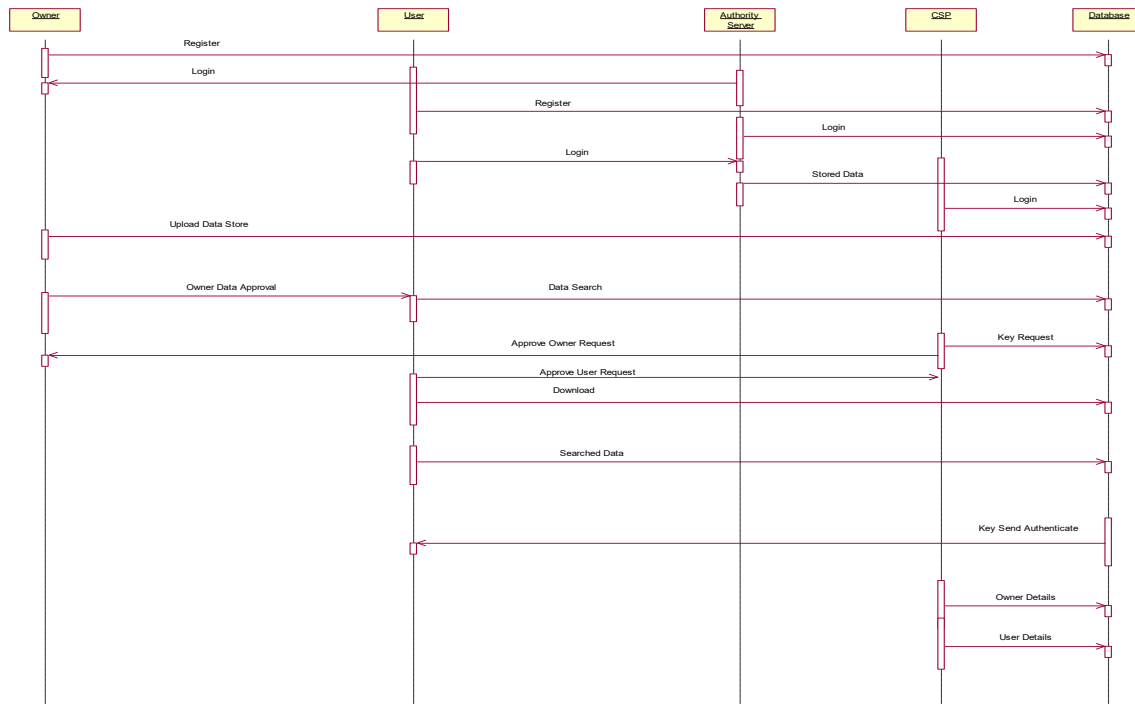


Fig.4.2.6.1 Sequence Diagram

EXPLANATION:

Sequence diagrams are graphical representations of sequence in manner it also has a The Four modules are available. Each detail is recorded in the owner's registration. It requested permission from the authority server to log in. Prior to logging in, the authority server must approve. Owner responds upon permission, and then it was time to log in. Owner has uploaded a text document as the data to be stored. Owner uploaded their own data. Owner views the data. Owner data is approved by the owner. A user's register contains information. The authority server also grants permissions to the user. The authority server has the ability to approve user requests. The database has a user-uploaded data search. Data was searched by the user. Additionally, users

can download data. The login for CSP requires a user ID and password. CSP has data that is stored. The owner and user details are collected by CSP. A user ID and password are required to log into the authority server. A key request was made to the authority server using a key generator. Owner and user requests are on the authority server.

4.2.7 COLLABORATION DIAGRAM

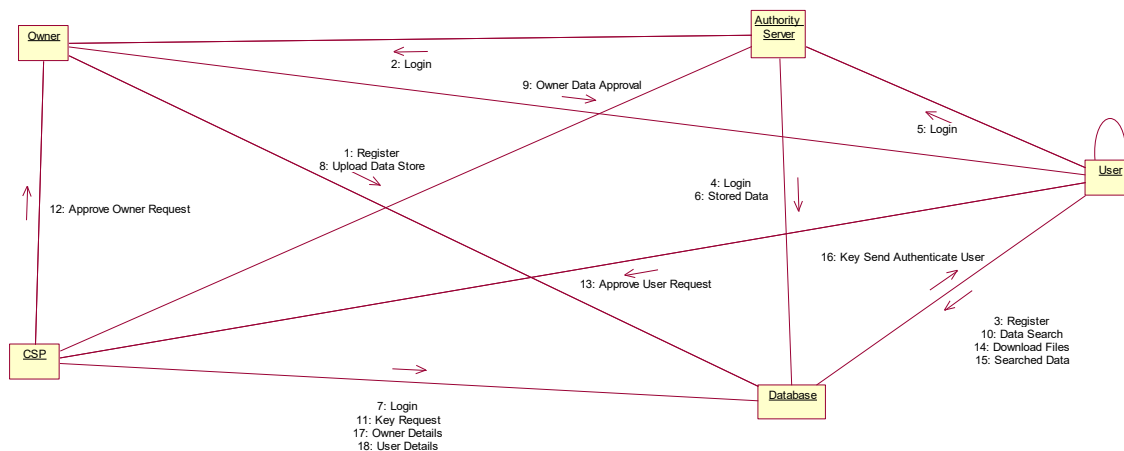


Fig.4.2.7.1 Collaboration Diagram

EXPLANATION:

A collaboration diagram, also called a communication diagram or interaction diagram it was also a evolved Information manager had a register with all the details in our project The Four modules are available. Each detail is recorded in the owner's registration. It requested permission from the authority server to log in. Prior to logging in, the authority server must approve. Owner responds upon permission, and then it was time to log in. Owner has uploaded a text document as the data to be stored. Owner uploaded their own data. Owner views the data. Owner data is approved by the owner. A user's register contains information. The authority server also grants permissions to the user. The authority server has the ability to approve user requests. The database has a user-uploaded data search. Data was searched by the user. Additionally, users can download data. The login for CSP requires a user ID and password. CSP has data that is stored. The owner and user details are collected by CSP. A user ID and password are required to log into the authority server.

A key request was made to the authority server using a key generator. Owner and user requests are on the authority server.

4.2.8 STATE DIAGRAM

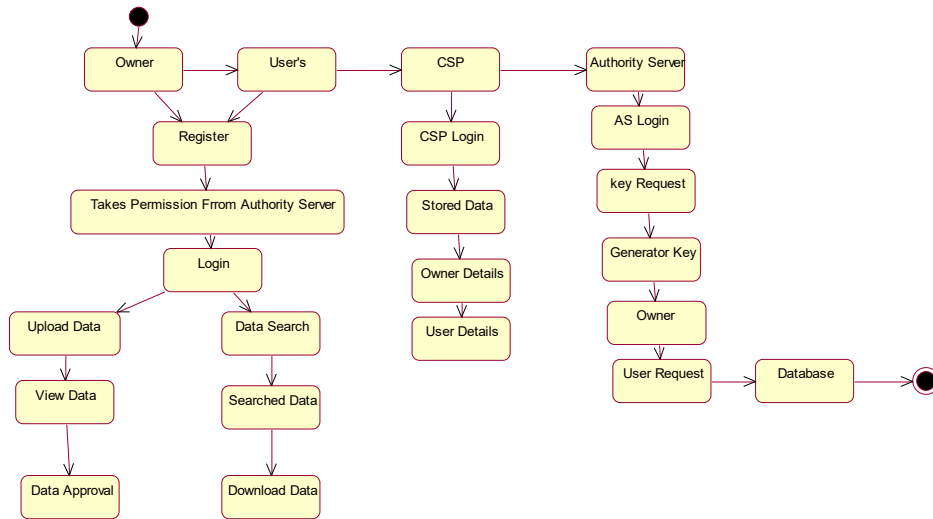


Fig.4.2.8.1 State Diagram

EXPLANATION:

State diagram are a loosely defined diagram it has a database. From the database it has a The Four modules are available. Each detail is recorded in the owner's registration. It requested permission from the authority server to log in. Prior to logging in, the authority server must approve. Owner responds upon permission, and then it was time to log in. Owner has uploaded a text document as the data to be stored. Owner uploaded their own data. Owner views the data. Owner data is approved by the owner. A user's register contains information. The authority server also grants permissions to the user. The authority server has the ability to approve user requests. The database has a user-uploaded data search. Data was searched by the user. Additionally, users can download data. The login for CSP requires a user ID and password. CSP has data that is stored. The owner and user details are collected by CSP. A user ID and password are required to log into the authority server. A key request was made to the authority server using a key generator. Owner and user requests are on the authority server.

4.2.9 ACTIVITY DIAGRAM

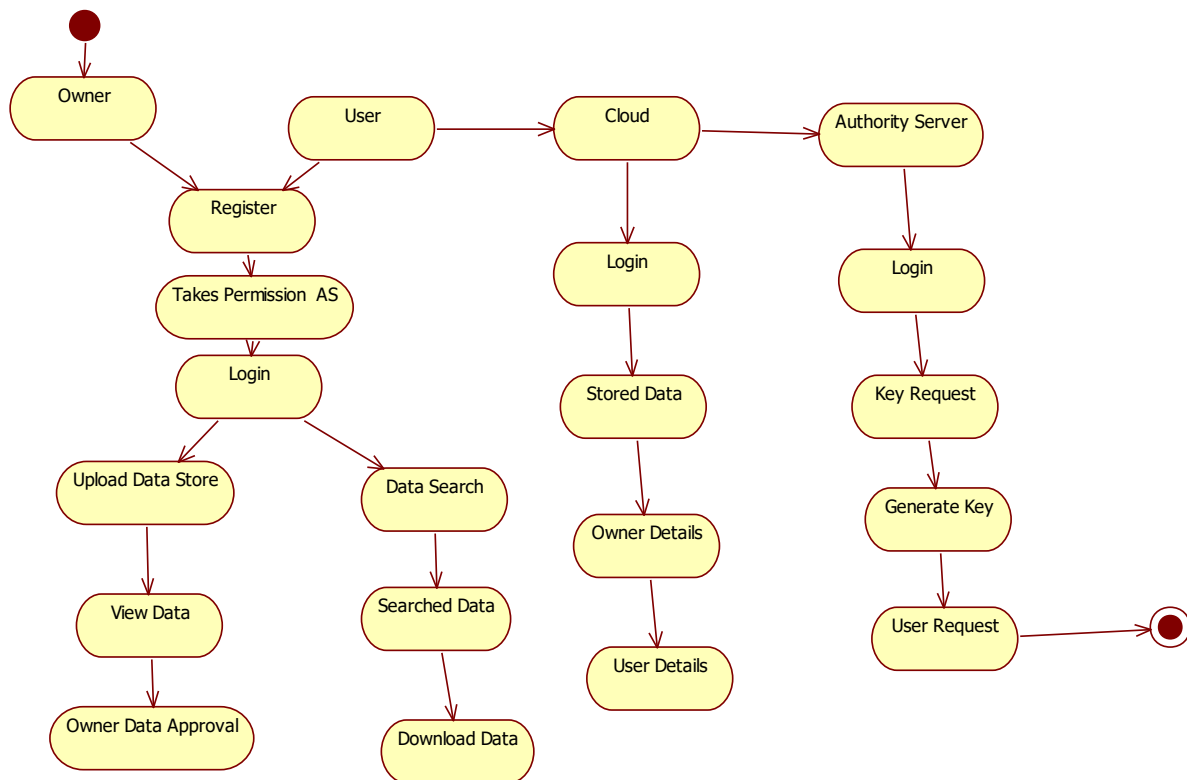


Fig.4.2.9.1 Activity Diagram

EXPLANATION:

Activity diagrams are graphical representations of workflows of stepwise activities and actions with support for choice. In the Unified Modeling Language, activity diagrams can be used to describe a The Four modules are available. Each detail is recorded in the owner's registration. It requested permission from the authority server to log in. Prior to logging in, the authority server must approve. Owner responds upon permission, and then it was time to log in. Owner has uploaded a text document as the data to be stored. Owner uploaded their own data. Owner views the data. Owner data is approved by the owner. A user's register contains information. The authority server also grants permissions to the user. The authority server has the ability to approve user requests. The database has a user-uploaded data search. Data was searched by the user. Additionally, users can download data. The login for CSP requires a user ID and password.CSP has data that is stored.

4.3 DATA FLOW DIAGRAM

Level 0

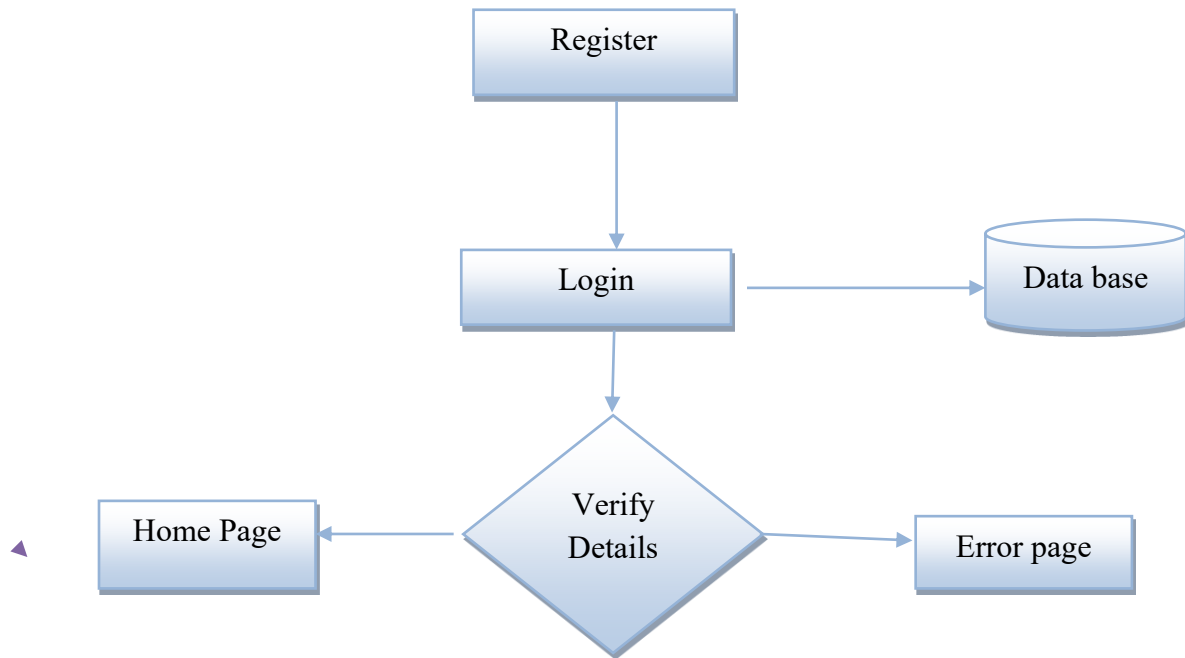


Fig.4.3.1 DFD Level 0

Level 1

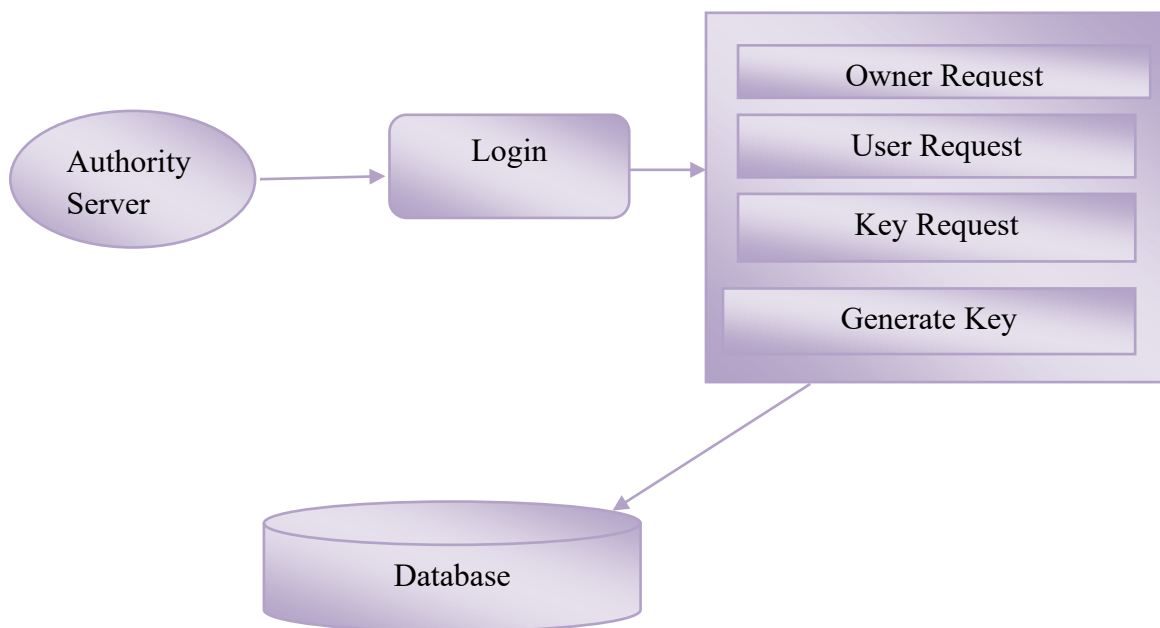


Fig.4.3.2 DFD Level 1

Level 2

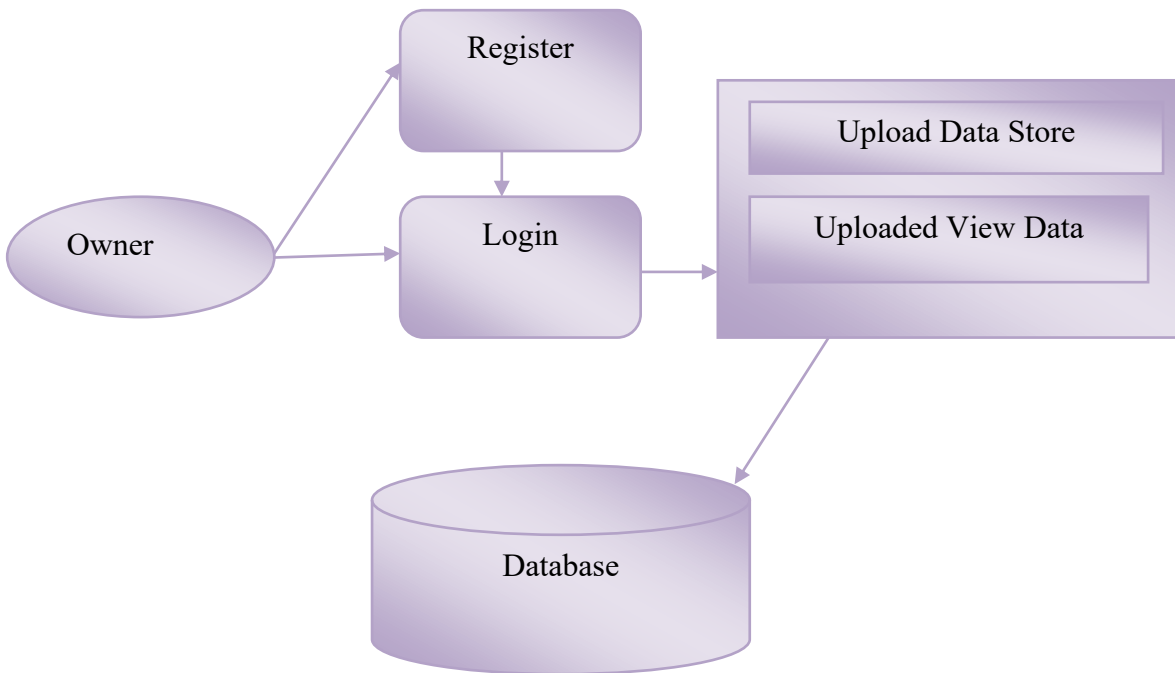


Fig 4.3.3 DFD Level 2

Level 3

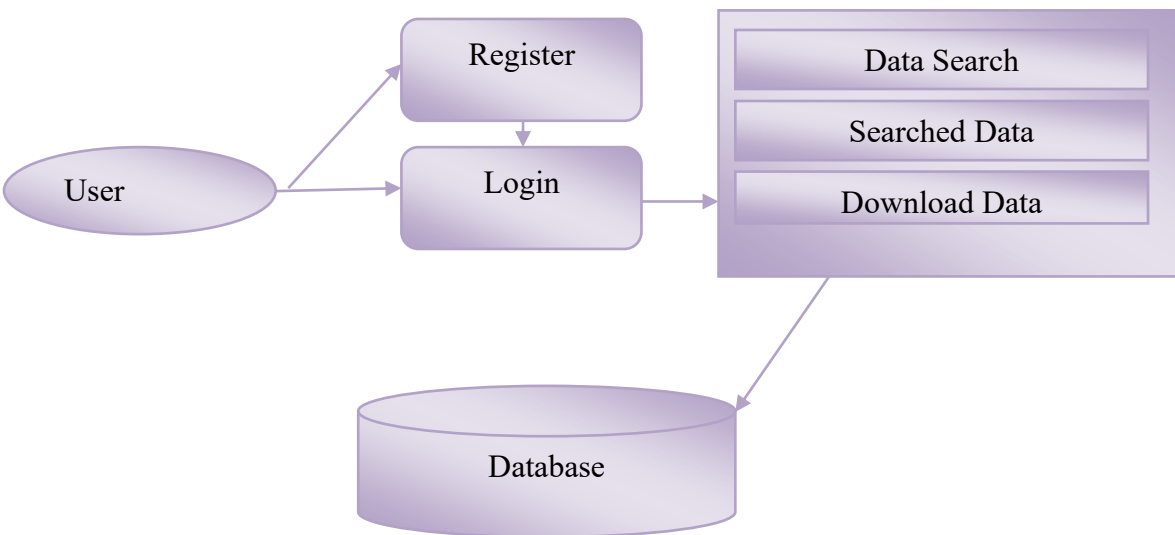


Fig.4.3.4 DFD Level 3

Level 4

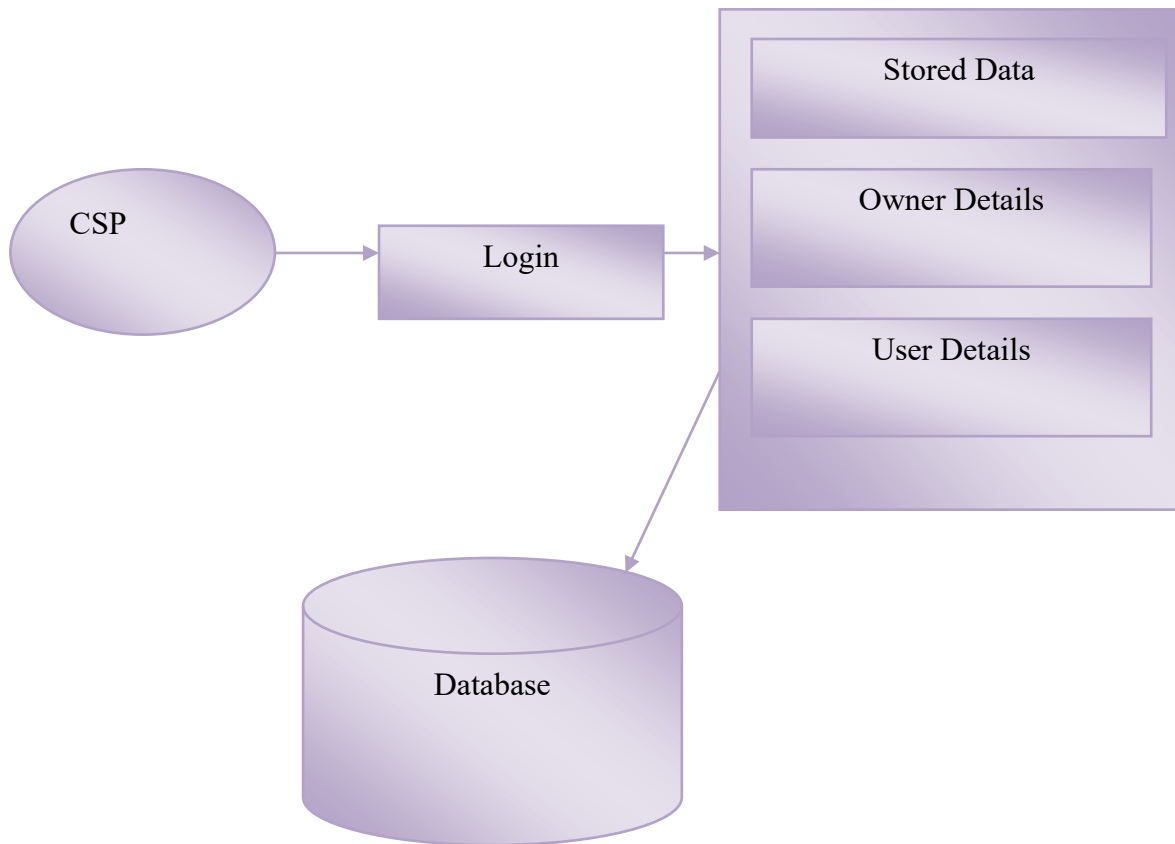


Fig.4.3.5 DFD Level 4

EXPLANATION:

A data flow diagram (DFD) is a graphical representation of the "flow" of data through an information system. It splits diagrams into four levels to easily understand a project diagram. The four modules are available. Each detail is recorded in the owner's registration. It requested permission from the authority server to log in. Prior to logging in, the authority server must approve. Owner responds upon permission, and then it was time to log in. Owner has uploaded a text document as the data to be stored. Owner uploaded their own data. Owner views the data. Owner data is approved by the owner. A user's register contains information. The authority server also grants permissions to the user. The authority server has the ability to approve user requests. A user ID and password are required to log into the authority server. A key request was made to the authority server using a key generator. Owner and user requests are on the authority server.

4.4 E-R DIAGRAM

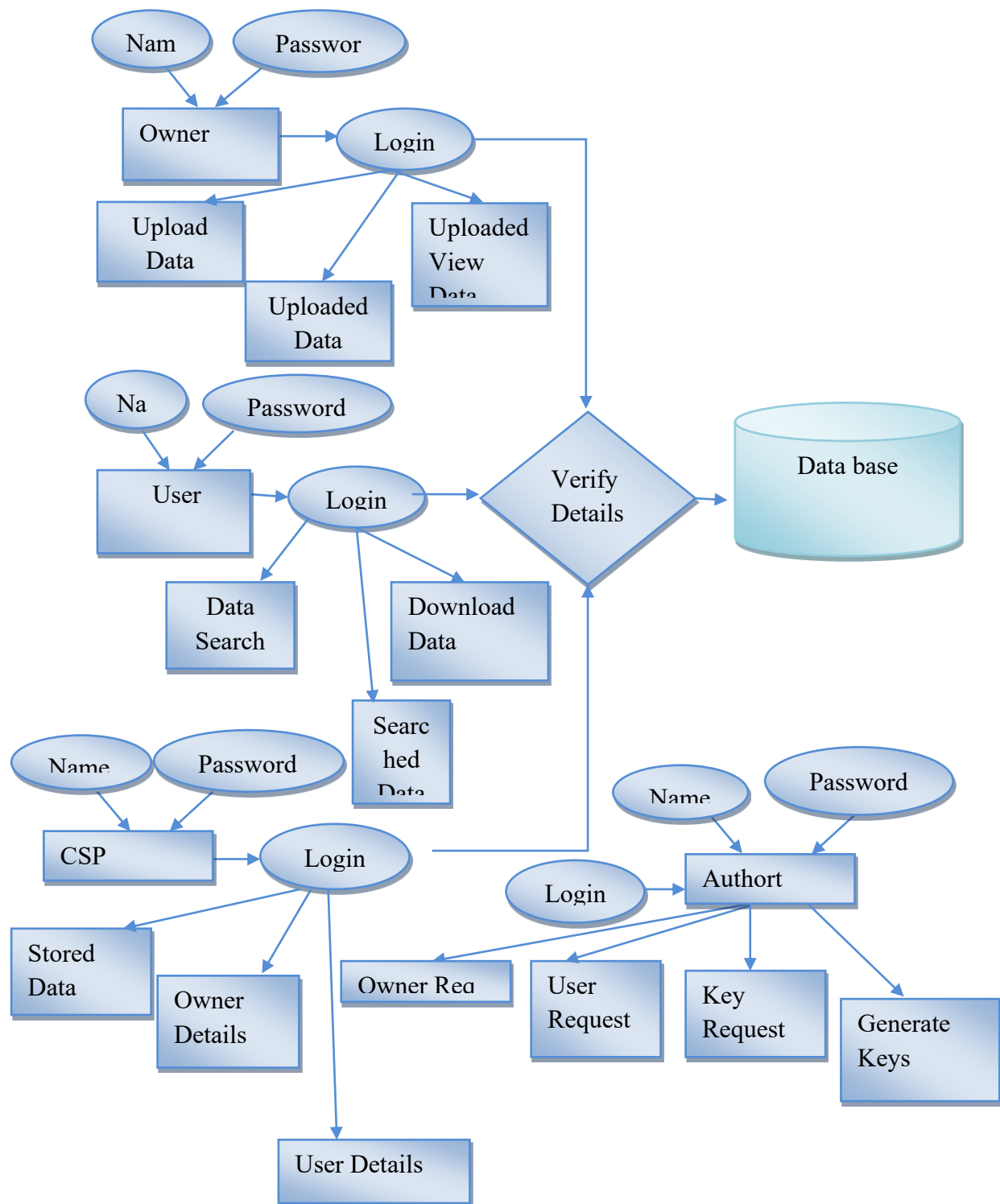


Fig.4.4.1 E-R Diagram

EXPLANATION:

Entity-Relationship Model (ERM) The Four modules are available. Each detail is recorded in the owner's registration. It requested permission from the authority server to log in. Prior to logging in, the authority server must approve. Owner responds upon permission, and then it was time to log in. Owner has uploaded a text document as the data to be stored. Owner uploaded their own data. Owner views the data. Owner data is approved by the owner. A user's register contains information. The authority server also grants permissions to the user. The authority server has the ability to approve user requests. The database has a user-uploaded data search. Data was searched by the user. Additionally, users can download data.

4.5 SYSTEM ARCHITECTURE

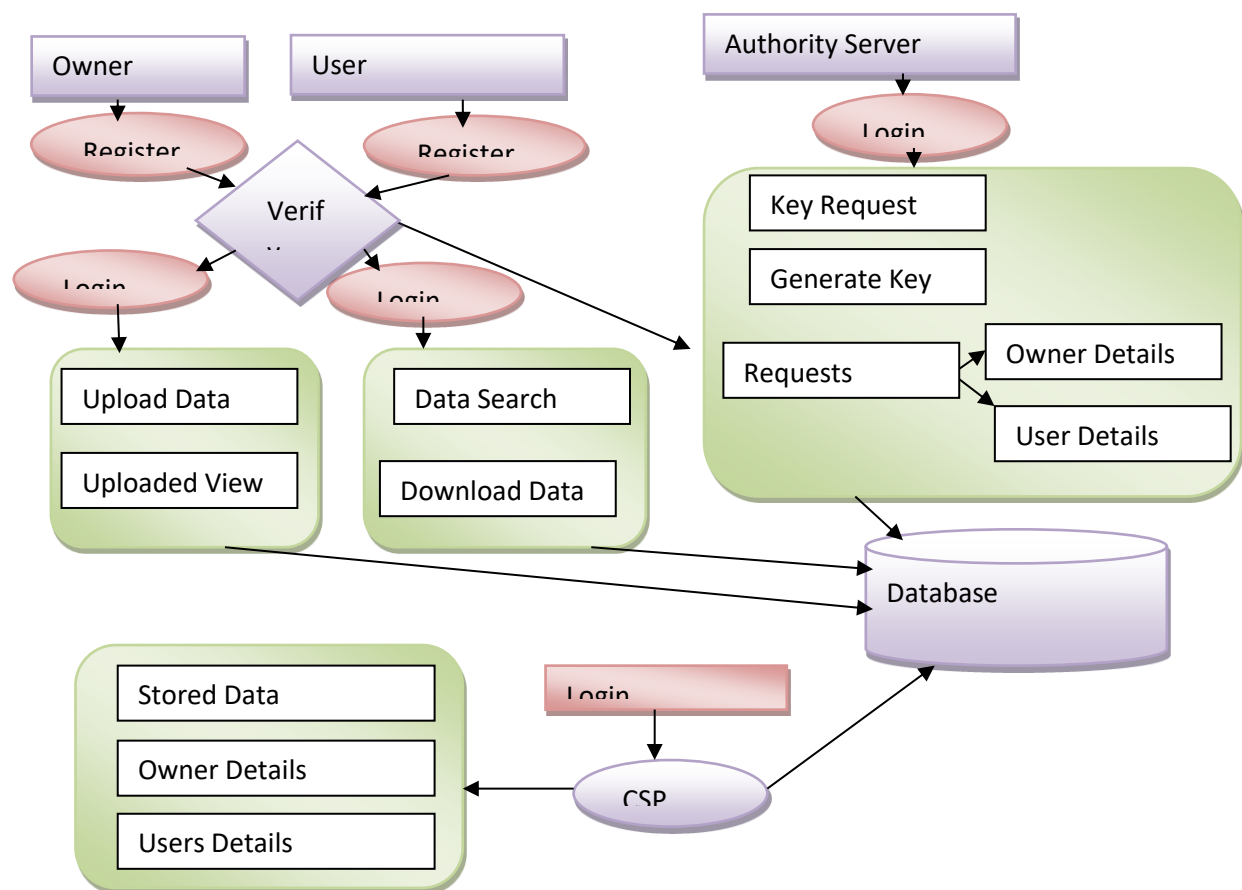


Fig.4.5.1 System Architecture

EXPLANATION

In this project we have a four modules. Owner has a register with all details. Login it was takes permission from the authority server. Authority server has to approves then it was login. Owner after approval it gives response then it was login. Owner has a upload a data to store a data it was upload a text document. Owner has a uploaded a own data .Owner has a views a data. Owner has a approves a owner data. User has a register with a details. User also takes permissions from the authority server. Authority server has a approves a requests user request it will approves. User has a data search uploaded in the database. User has a searched a data. User also have a download a data.CSP has a login with a user id and password.CSP has a stored a data.CSP has a get the details of the owner and user details. Authority server has a login with a user id and password. Authority server has a key request it was a generate key generator. Authority server has a requests of a owner and user requests.

CHAPTER 5

DEVELOPMENT TOOLS

5.1 GENERAL

This chapter is about the software language and the tools used in the development of the project. The platform used here is JAVA. The Primary languages are JAVA,J2EE and J2ME. In this project J2EE is chosen for implementation.

5.2 FEATURES OF JAVA

5.2.1 THE JAVA FRAMEWORK

Java is a programming language originally developed by James Gosling at Microsystems and released in 1995 as a core component of Sun Microsystems' Java platform. The language derives much of its syntax from C and C++ but has a simpler object model and fewer low-level facilities. Java applications are typically compiled to byte code that can run on any Java Virtual Machine (JVM) regardless of computer architecture. Java is general-purpose, concurrent, class-based, and object-oriented, and is specifically designed to have as few implementation dependencies as possible. It is intended to let application developers "write once, run anywhere".

Java is considered by many as one of the most influential programming languages of the 20th century, and is widely used from application software to web applications the java framework is a new platform independent that simplifies application development internet. Java technology's versatility, efficiency, platform portability, and security make it the ideal technology for network computing. From laptops to datacenters, game consoles to scientific supercomputers, cell phones to the Internet, Java is everywhere!

5.2.2 OBJECTIVES OF JAVA

To see places of Java in Action in our daily life, explore java.com.

WHY SOFTWARE DEVELOPERS CHOOSE JAVA

Java has been tested, refined, extended, and proven by a dedicated community. And numbering more than 6.5 million developers, it's the largest and most active on the planet. With its versatility, efficiency, and portability, Java has become invaluable to developers by enabling them to:

- Write software on one platform and run it on virtually any other platform
- Create programs to run within a Web browser and Web services

- Develop server-side applications for online forums, stores, polls, HTML forms processing, and more
- Combine applications or services using the Java language to create highly customized applications or services
- Write powerful and efficient applications for mobile phones, remote processors, low-cost consumer products, and practically any other device with a digital heartbeat

SOME WAYS SOFTWARE DEVELOPERS LEARN JAVA

Today, many colleges and universities offer courses in programming for the Java platform. In addition, developers can also enhance their Java programming skills by reading Sun's java.sun.com Web site, subscribing to Java technology-focused newsletters, using the Java Tutorial and the New to Java Programming Center, and signing up for Web, virtual, or instructor-led courses.

OBJECT ORIENTED

To be an Object Oriented language, any language must follow at least the four characteristics.

1. Inheritance :It is the process of creating the new classes and using the behavior of the existing classes by extending them just to reuse the existing code and adding additional features as needed.
2. Encapsulation: It is the mechanism of combining the information and providing the abstraction.
3. Polymorphism: As the name suggests one name multiple form, Polymorphism is the way of providing the different functionality by the functions having the same name based on the signatures of the methods.

5.2.3 JAVA SWING OVERVIEW

ABSTRACT WINDOW TOOLKIT (AWT) IS CROSS-PLATFORM

Swing provides many controls and widgets to build user interfaces with. Swing class names typically begin with a J such as JButton, JList, JFrame. This is mainly to differentiate them from their AWT counterparts and in general is one-to-one replacements. Swing is built on the concept of Lightweight components vs AWT and SWT's concept of Heavyweight components.

NOTE: It generally it is not a good idea to mix heavyweight components with lightweight components (other than as previously mentioned) as you will encounter layering issues, e.g., a lightweight component that should appear "on top" ends up being obscured by a heavyweight component. The few exceptions to this include using heavyweight components as the root pane

and for popup windows. The truly adventurous may want to consider reading this article from Sun on mixing heavyweight and lightweight components.

5.2.4 EVOLUTION OF COLLECTION FRAMEWORK:

Almost all collections in Java are derived from the `java.util.Collection` interface. `Collection` defines the basic parts of all collections. The interface states the `add()` and `remove()` methods for adding to and removing from a collection respectively. Also required is the `toArray()` method, which converts the collection into a simple array of all the elements in the collection. Finally, the `contains()` method checks if a specified element is in the collection. For example, `Collection<String>` can hold strings, and the elements from the collection can be used as strings without any casting required.

There are three main types of collections:

- Lists: always ordered, may contain duplicates and can be handled the same way as usual arrays
- Sets: cannot contain duplicates and provide random access to their elements
- Maps: connect unique keys with values, provide random access to its keys and may host duplicate values

LIST

Lists are implemented in the JCF via the `java.util.List` interface. It defines a list as essentially a more flexible version of an array. Elements have a specific order, and duplicate elements are allowed. Elements can be placed in a specific position. They can also be searched for within the list. Two concrete classes implement `List`. The first is `java.util.ArrayList`, which implements the list as an array. The other implementation is `java.util.LinkedList`.

SET:

Java's `java.util.Set` interface defines the set. A set can't have any duplicate elements in it. Additionally, the set has no set order. As such, elements can't be found by index. `Set` is implemented by `java.util.HashSet`, `java.util.LinkedHashSet`, and `java.util.TreeSet`. `HashSet` uses a hash table. More specifically, it uses a `java.util.HashMap` to store the hashes and elements and to prevent duplicates. The `java.util.Set` interface is extended by the `java.util.SortedSet` interface. Unlike a regular set, the elements in a sorted set are sorted, either by the element's `compareTo()` method, or a method provided to the constructor of the sorted set.

MAP:

Maps are defined by the `java.util.Map` interface in Java. Maps are simple data structures that associate a key with a value. The element is the value. The hashes of the keys are used to find the values in various buckets. `LinkedHashMap` extends this by creating a doubly linked list between the elements. This allows the elements to be accessed in the order in which they were inserted into the map. `TreeMap`, in contrast to `HashMap` and `LinkedHashMap`, uses a red-black tree. The keys are used as the values for the nodes in the tree, and the nodes point to the values in the map

THREAD:

Simply put, a *thread* is a program's path of execution. Most programs written today run as a single thread, causing problems when multiple events or actions need to occur at the same time. Let's say, for example, a program is not capable of drawing pictures while reading keystrokes. The program must give its full attention to the keyboard input lacking the ability to handle more than one event at a time. The ideal solution to this problem is the seamless execution of two or more sections of a program at the same time.

CREATING THREADS

Java's creators have graciously designed two ways of creating threads: implementing an interface and extending a class. This limitation within Java can be overcome by implementing interfaces, which is the most common way to create threads. (Note that the act of inheriting merely allows the class to be run as a thread. It is up to the class to `start()` execution, etc.)

Interfaces provide a way for programmers to lay the groundwork of a class. They are used to design the requirements for a set of classes to implement.

5.5 CONCLUSION

Swing's high level of flexibility is reflected in its inherent ability to override the native host operating system (OS)'s GUI controls for displaying itself. Swing "paints" its controls using the Java 2D APIs, rather than calling a native user interface toolkit. The Java thread scheduler is very simple. All threads have a priority value which can be changed dynamically by calls to the `threads setPriority()` method . Implementing the above concepts in our project to do the efficient work among the Server.

CHAPTER 6

IMPLEMENTATION

6.1 GENERAL

CODING:

```
<%@ page import="java.sql.*" %>

<html>

<head>

<title>Owner Page</title>

</head>

<body>

<h2>Owner Registration</h2>

<form method="post">

Name: <input type="text" name="name" required><br><br>

Email: <input type="text" name="email" required><br><br>

Age: <input type="text" name="age" required><br><br>

Gender: <input type="text" name="gender" required><br><br>

Password: <input type="password" name="pwd" required><br><br>

<input type="submit" name="register" value="Register">

</form>
```

<hr>

<h2>Owner Login</h2>

<form method="post">

Email: <input type="text" name="lemail">

Password: <input type="password" name="lpwd">

<input type="submit" name="login" value="Login">

</form>

<%

String url = "jdbc:mysql://localhost:3306/test";

String user = "root";

String pass = "root";

Class.forName("com.mysql.jdbc.Driver");

Connection con = DriverManager.getConnection(url, user, pass);

if(request.getParameter("register") != null){

String name = request.getParameter("name");

String email = request.getParameter("email");

String age = request.getParameter("age");

String gender = request.getParameter("gender");

String pwd = request.getParameter("pwd");

```

        PreparedStatement ps = con.prepareStatement(

            "INSERT INTO owner(name,email,age,gender,password) VALUES(?,?,?,?)"

        );

        ps.setString(1,name);

        ps.setString(2,email);

        ps.setString(3,age);

        ps.setString(4,gender);

        ps.setString(5,pwd);

        int i = ps.executeUpdate();

        if(i>0){

            out.println("<script>alert('Registered Successfully');</script>");

        }

    }

    if(request.getParameter("login") != null){

        String email = request.getParameter("lemail");

        String pwd = request.getParameter("lpwd");

        PreparedStatement ps = con.prepareStatement(

            "SELECT * FROM owner WHERE email=? AND password=?"

        );

```

```
ps.setString(1,email);

ps.setString(2,pwd);

ResultSet rs = ps.executeQuery();

if(rs.next()){

    response.sendRedirect("ownerDetails.jsp");

} else {

    out.println("<script>alert('Invalid Login');</script>");

}

%>

</body>

</html>
```

SQL:

```
CREATE TABLE owner (

    name VARCHAR(50),

    email VARCHAR(50),

    age VARCHAR(10),

    gender VARCHAR(10),

)
```

CHAPTER 7

SNAPSHOTS

7.1 GENERAL:

This project implements a web application using COREJAVA and the Server process is maintained using the SOCKET & SERVERSOCKET and the Design part is played by Cascading Style Sheet.

7.2 SNAPSHOTS

The screenshot shows the 'DataMining' website interface. At the top, there's a header with contact information (contact@example.com, +91 9581022022) and social media icons. Below the header is a navigation bar with links: Home, Owner, User's, CSP, AUTHORITY SERVER, and Contact. The main content area is titled 'OWNER' and shows a 'Registration' form. The form fields are: Name (Sravanthi), Email (sravanthi@gmail.com), Age (22), Gender (Female), and PASSWORD (Sravanthi@123). There are 'Register' and 'Reset' buttons at the bottom of the form. A green checkmark icon is visible next to the password field.

Figure 7.2.1 Owners Registration

The screenshot shows the 'DataMining' website interface. At the top, there's a header with contact information (contact@example.com, +91 9581022022) and social media icons. Below the header is a navigation bar with links: Home, Owner, User's, CSP, AUTHORITY SERVER, and Contact. The main content area is titled 'USER' and shows a 'User Register' form. The form fields are: NAME (Poojitha), EMAIL ID (poojitha@gmail.com), AGE (22), GENDER (Female), and PASSWORD (masked with dots). There are 'Register' and 'Reset' buttons at the bottom of the form. A green checkmark icon is visible next to the password field. Below the form, there is a link for 'User Login'.

Figure 7.2.2 User's Registration

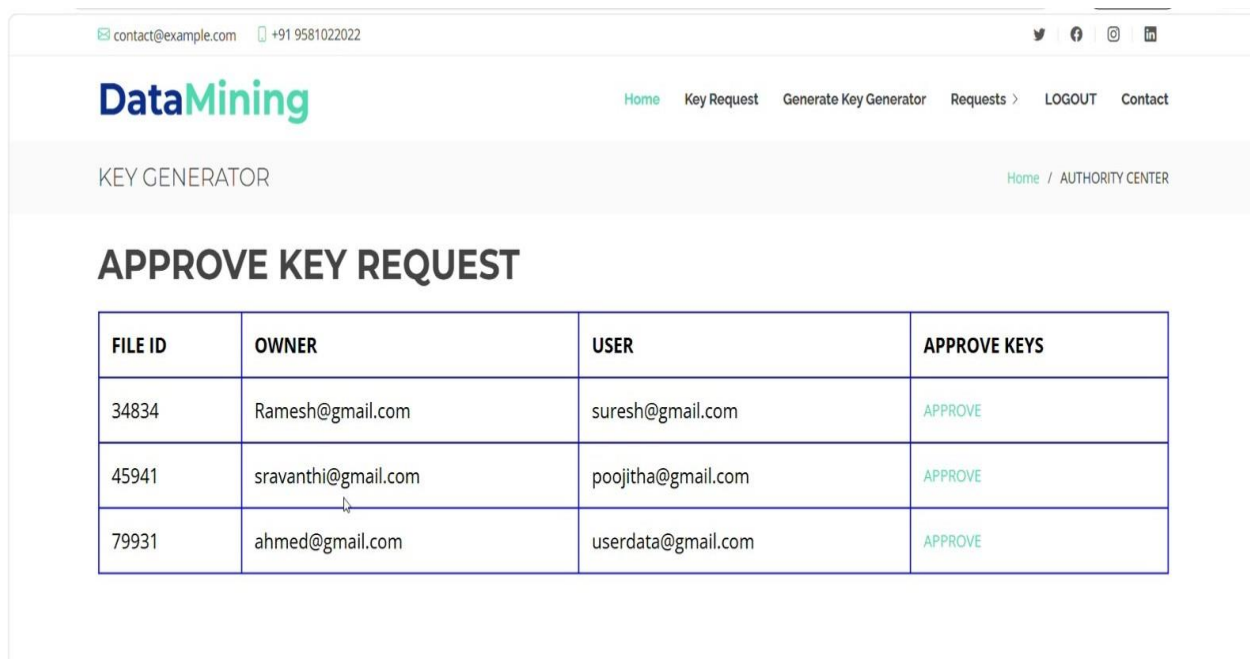


Figure 7.2.3 Approving Key Requests

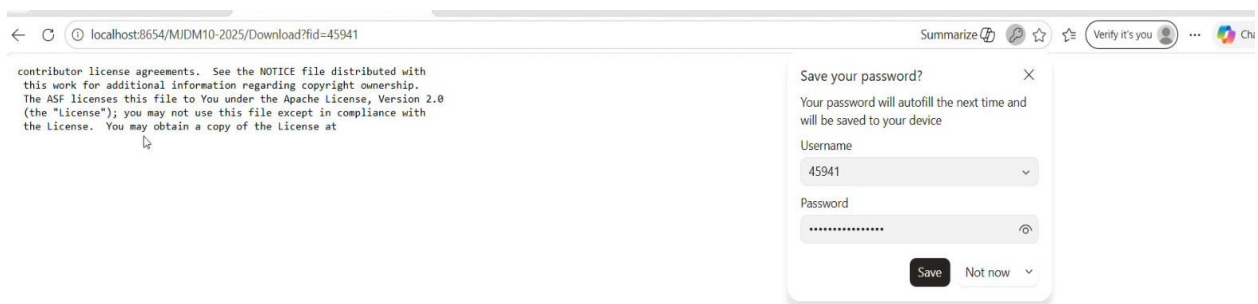


Figure 7.2.4 Decrypted Data

CHAPTER 8

SOFTWARE TESTING

8.1 GENERAL

The purpose of testing is to discover errors. Testing is the process of trying to discover every conceivable fault or weakness in a work product. It provides a way to check the functionality of components, sub assemblies, assemblies and/or a finished product. It is the process of exercising software with the intent of ensuring that the Software system meets its requirements and user expectations and does not fail in an unacceptable manner. There are various types of test. Each test type addresses a specific testing requirement.

8.2 DEVELOPING METHODOLOGIES

The test process is initiated by developing a comprehensive plan to test the general functionality and special features on a variety of platform combinations. Strict quality control procedures are used. The process verifies that the application meets the requirements specified in the system requirements document and is bug free. The following are the considerations used to develop the framework from developing the testing methodologies.

8.3 TYPES OF TESTS

Sl. No	Test scenario	User action	Expected result	Actual Result	Remarks
1.	Registration	Users registering into the system.	Register into the system.	Successfully alert registered message.	Pass
2.	Login	1. Entered correct password.	1. Log into the system. 2. Alert generated.	1. Successfully logged in.	Pass

				2. Successfully generated the alert.	
3.	Authority server	Generate tk and shows all information	Messages sending server has got alert is generated.	Successfully generated the alert and messages sending	Successful
4.	Owner	Upload a data and view all file information.	Owner has to actions	Successfully generated the alert to Owner message	Successful
5.	User	See all files and download.	Messages Alert is generated	Successfully generated the alert for in User messages	Successful
6.	CSP	We have a stored data information and details.	Messages Alert is generated	Successfully generated the alert for CSP messages	Successful

8.3.1 UNIT TESTING

Unit testing involves the design of test cases that validate that the internal program logic is functioning properly, and that program input produce valid outputs. All decision branches and internal code flow should be validated. It is the testing of individual software units of the application .it is done after the completion of an individual unit before integration. This is a structural testing, that relies on knowledge of its construction and is invasive. Unit tests perform basic tests at component level and test a specific business process, application, and/or system

configuration. Unit tests ensure that each unique path of a business process performs accurately to the documented specifications and contains clearly defined inputs and expected results.

8.3.2 FUNCTIONAL TEST

Functional tests provide systematic demonstrations that functions tested are available as specified by the business and technical requirements, system documentation, and user manuals.

Functional testing is centered on the following items:

- Valid Input : identified classes of valid input must be accepted.
- Invalid Input : identified classes of invalid input must be rejected.
- Functions : identified functions must be exercised.
- Output : identified classes of application outputs must be exercised.
- Systems/Procedures : interfacing systems or procedures must be invoked.

8.3.3 SYSTEM TEST

System testing ensures that the entire integrated software system meets requirements. It tests a configuration to ensure known and predictable results. An example of system testing is the configuration oriented system integration test. System testing is based on process descriptions and flows, emphasizing pre-driven process links and integration points.

8.3.4 PERFORMANCE TEST

The Performance test ensures that the output be produced within the time limits, and the time taken by the system for compiling, giving response to the users and request being send to the system for to retrieve the results.

8.3.5 INTEGRATION TESTING

Software integration testing is the incremental integration testing of two or more integrated software components on a single platform to produce failures caused by interface defects.

The task of the integration test is to check that components or software applications, e.g. components in a software system or – one step up – software applications at the company level – interact without error.

8.3.6 ACCEPTANCE TESTING

User Acceptance Testing is a critical phase of any project and requires significant participation by the end user. It also ensures that the system meets the functional requirements.

ACCEPTANCE TESTING FOR DATA SYNCHRONIZATION:

- The Acknowledgements will be received by the Sender Node after the Packets are received by the Destination Node
- The Route add operation is done only when there is a Route request in need
- The Status of Nodes information is done automatically in the Cache Updating process

8.2.7 BUILD THE TEST PLAN

Any project can be divided into units that can be further performed for detailed processing. Then a testing strategy for each of this unit is carried out. Unit testing helps to identify the possible bugs in the individual component, so the component that has bugs can be identified and can be rectified from errors.

CHAPTER 9

9.1 CONCLUSION

Based on our review of literature on deniable encryption schemes, the user can, in past schemes, come up with only one fake message as a counterpart to defend its secret. In contrast, our scheme allows us to bring more than one fake message to hide a secret. Also, this encryption scheme is editable in the sense that at any instant in time, we can update our message by changing only one element in the cipher text or by changing the key indices. Furthermore, the scheme gives us the freedom to delete any message encrypted inside the cipher text simply by replacing at least one element from the cipher text with a random number. Deleting a message gives us the flexibility to prevent the user who was earlier accessing that message from doing it again. If the user wants to decrypt the cipher text with the older key, the outcome will undoubtedly dissatisfy him. Consequently, False-Bottom Encryption extends deniable encryption by the functionality of adding, editing and deleting possibly several plaintexts inside the cipher text. Our security definition does not account for adversaries profiling the access patterns of a user, which calls for additional techniques to either randomize or “equalize” all access sequences.

9.2 FUTURE ENHANCEMENT

Future work will thus investigate extensions to our scheme by means of private information retrieval or other techniques (see the related work, in particular), to analyze if information-theoretic security remains accomplishable or deteriorates against attackers that profile the (physical) device usage.

9.3 REFERENCES

- [1] R. Canetti, U. Feige, O. Goldreich, and M. Naor, “Adaptively secure multi-party computation,” in *Proc. 28th Annu. ACM Symp. Theory Comput.*, Philadelphia, PA, USA, 1996, pp. 639–648. [Online]. Available: <http://portal.acm.org/citation.cfm?doid=237814.238015>
- [2] R. Canetti, C. Dwork, M. Naor, and R. Ostrovsky, “Deniable encryption,” in *Proc. 17th Annu. Int. Cryptol. Conf.* (Lecture Notes in Computer Science), vol. 1294. Santa Barbara, CA, USA: Springer, 1997, pp. 90–104.
- [3] A. Juels and T. Ristenpart, “Honey encryption: Security beyond the brute-force bound,” in *Advances in Cryptology—EUROCRYPT* (Lecture Notes in Computer Science), vol. 8441, D. Hutchison, T. Kanade,
- [4] J. Kittler, J. M. Kleinberg, A. Kobsa, F. Mattern, J. C. Mitchell, M. Naor, O. Nierstrasz, C. Pandu Rangan, B. Steffen, D. Terzopoulos, D. Tygar, G. Weikum, P. Q. Nguyen, and E. Oswald, Eds. Berlin, Germany: Springer, 2014, pp. 293–310, doi: 10.1007/978-3-642-55220-5_17.
- [5] M. Durmuth and D. M. Freeman, “Deniable encryption with negligible detection probability: An interactive construction,” in *Advances in Cryptology—EUROCRYPT* (Lecture Notes in Computer Science), vol. 6632, D. Hutchison, T. Kanade, J. Kittler, J. M. Kleinberg, F. Mattern,
- [6] J. C. Mitchell, M. Naor, O. Nierstrasz, C. Pandu Rangan, B. Steffen, M. Sudan, D. Terzopoulos, D. Tygar, M. Y. Vardi, G. Weikum, and K. G. Paterson, Eds. Berlin, Germany: Springer, 2011, pp. 610–626, doi: 10.1007/978-3-642-20465-4_33.
- [7] A. O’Neill, C. Peikert, and B. Waters, “Bi-deniable public-key encryption,” in *Advances in Cryptology—CRYPTO* (Lecture Notes in Computer Science), vol. 6841, D. Hutchison, T. Kanade, J. Kittler, J. M. Kleinberg,
- [8] F. Mattern, J. C. Mitchell, M. Naor, O. Nierstrasz, C. P. Rangan, B. Steffen, M. Sudan, D. Terzopoulos, D. Tygar, M. Y. Vardi, G. Weikum, and P. Rogaway, Eds. Berlin, Germany: Springer, 2011, pp. 525–542, doi: 10.1007/978-3-642-22792-9_30.
- [9] M. Klonowski, P. Kubiak, and M. Kutylowski, “Practical deniable encryption,” in *SOFSEM 2008: Theory and Practice of Computer Science* (Lecture Notes in Computer Science), V. Geffert, J. Karhumäki, A. Bertoni, B. Preneel, P. Návrat, and M. Bieliková, Eds. Berlin, Germany: Springer, 2008, pp. 599–609.
- [10] P. Gasti, G. Ateniese, and M. Blanton, “Deniable cloud storage: sharing files via public-key

- deniability,” in *Proc. 9th Annu. ACM Workshop Privacy Electron. Soc.*, Chicago, IL, USA, 2010, p. 31. [Online]. Available: <http://portal.acm.org/citation.cfm?doid=1866919.1866925>
- [11] M. H. Ibrahim, “A method for obtaining deniable public-key encryption,” *Int. J. Netw. Secur.*, vol. 8, no. 1, pp. 1–9, 2009.
- [12] P. Chi and C. Lei, “Audit-free cloud storage via deniable attribute-based encryption,” *IEEE Trans. Cloud Comput.*, vol. 6, no. 2, pp. 414–427, Apr. 2018. [Online]. Available: <https://ieeexplore.ieee.org/document/7090980/>
- [13] R. Canetti, S. Park, and O. Poburinnaya, “Fully deniable interactive encryption,” in *Advances in Cryptology—CRYPTO* (Lecture Notes in Computer Science), vol. 12170, D. Micciancio and T. Ristenpart, Eds. Cham, Switzerland: Springer, 2020, pp. 807–835, doi: 10.1007/978-3-030-56784-2_27.
- [14] C. Dwork, M. Naor, and A. Sahai, “Concurrent zero-knowledge,” *J. ACM*, vol. 51, no. 6, p. 851–898, Nov. 2004, doi: [10.1145/1039488.1039489](https://doi.org/10.1145/1039488.1039489).
- [15] M. Naor, “Deniable ring authentication,” in *Advances in Cryptology—CRYPTO* (Lecture Notes in Computer Science), vol. 2442, G. Goos, Ed. Berlin, Germany: Springer, 2002, pp. 481–498, doi: [10.1007/3-540-45708-9_31](https://doi.org/10.1007/3-540-45708-9_31).
- [16] R. W. Zhu, D. S. Wong, and C. H. Lee, “Cryptanalysis of a suite of deniable authentication protocols,” *IEEE Commun. Lett.*, vol. 10, no. 6, pp. 504–506, Jun. 2006.
- [17] A. Fiat and M. Naor, “Broadcast encryption,” in *Proc. Annu. Int. Cryptol. Conf.*, Jan. 1993, pp. 480–491.
- [18] J. Li, Y. Wang, Y. Zhang, and J. Han, “Full verifiability for out-sourced decryption in attribute based encryption,” *IEEE Trans. Services Comput.*, vol. 13, no. 3, pp. 478–487, May 2020. [Online]. Available: <https://ieeexplore.ieee.org/document/7936626/>
- [19] J. Li, Q. Yu, and Y. Zhang, “Hierarchical attribute based encryption with continuous leakage-resilience,” *Inf. Sci.*, vol. 484, pp. 113–134, May 2019. [Online]. Available: <https://linkinghub.elsevier.com/retrieve/pii/S0020025519300684>